国际学生单车租赁系统

版本号：V1.0

源代码

目录

1. 国际学生单车租赁系统源代码
2. 目录
3. 1 源码选择说明
4. 源码文件：`backend/src/server.js`
5. 源码文件：`backend/src/app.js`
6. 源码文件：`backend/src/config/db.js`
7. 源码文件：`backend/src/middleware/auth.js`
8. 源码文件：`backend/src/middleware/role.js`
9. 源码文件：`backend/src/routes/auth.js`
10. 源码文件：`backend/src/routes/bikes.js`
11. 源码文件：`backend/src/routes/orders.js`
12. 源码文件：`backend/src/routes/stations.js`
13. 源码文件：`backend/src/routes/ai.js`
14. 源码文件：`backend/src/ai/agent.js`
15. 源码文件：`backend/src/ai/provider/base.js`
16. 源码文件：`backend/src/ai/provider/ollama.js`
17. 源码文件：`backend/src/ai/tools/index.js`
18. 源码文件：`frontend/src/App.vue`
19. 源码文件：`frontend/src/router/index.js`
20. 源码文件：`frontend/src/stores/auth.js`
21. 源码文件：`frontend/src/stores/ai.js`
22. 源码文件：`frontend/src/components/ChatWidget.vue`
23. 源码文件：`frontend/src/views/Login.vue`
24. 源码文件：`frontend/src/views/Bikes.vue`
25. 源码文件：`frontend/src/views/Orders.vue`
26. 源码文件：`frontend/src/assets/style.css`
27. 源码文件：`android/app/src/main/java/com/xuelinhu/lanwebclient/MainActivity.java`
28. 源码文件：`backend/postgres/schema.sql`
29. 源码文件：`backend/postgres/migrations/005_models_payments_i18n.js`

国际学生单车租赁系统源代码

目录

1. 源码选择说明
2. backend/src/server.js
3. backend/src/app.js
4. backend/src/config/db.js
5. backend/src/middleware/auth.js
6. backend/src/middleware/role.js
7. backend/src/routes/auth.js
8. backend/src/routes/bikes.js
9. backend/src/routes/orders.js
10. backend/src/routes/stations.js
11. backend/src/routes/ai.js
12. backend/src/ai/agent.js
13. backend/src/ai/provider/base.js
14. backend/src/ai/provider/ollama.js
15. backend/src/ai/tools/index.js
16. frontend/src/App.vue
17. frontend/src/router/index.js
18. frontend/src/stores/auth.js
19. frontend/src/stores/ai.js
20. frontend/src/components/ChatWidget.vue
21. frontend/src/views/Login.vue
22. frontend/src/views/Bikes.vue
23. frontend/src/views/Orders.vue
24. frontend/src/assets/style.css
25. android/app/src/main/java/com/xuelinhu/lanwebclient/MainActivity.java
26. backend/postgres/schema.sql
27. backend/postgres/migrations/005_models_payments_i18n.js

1 源码选择说明

以下内容来自当前提交的自研业务代码，排除了 node_modules、锁文件、构建产物和第三方源码。代码顺序与仓库相对路径保持一致。

源码文件：backend/src/server.js

SHA-256：0054d596dbd16f75dca78c50f0fdf29a159e1c53e0782900822e00783fca65e2

```
import app from './app.js';

const port = Number(process.env.PORT || 8032);
const host = process.env.API_HOST || '127.0.0.1';
app.listen(port, host, () => {
  console.log(`Student bike rental API running at http://${host}:${port}`);
});
```

源码文件：backend/src/app.js

SHA-256：0303eef6192a0faf25b16e92913fc6cd691fa6bc6ee5d578f18855e0eff0a4af

```
import express from 'express';
import cors from 'cors';
import dotenv from 'dotenv';
import authRoutes from './routes/auth.js';
import userRoutes from './routes/users.js';
import bikeRoutes from './routes/bikes.js';
import stationRoutes from './routes/stations.js';
import orderRoutes from './routes/orders.js';
import maintenanceRoutes from './routes/maintenance.js';
import announcementRoutes from './routes/announcements.js';
import dashboardRoutes from './routes/dashboard.js';
import aiRoutes from './routes/ai.js';
import modelRoutes from './routes/models.js';
import paymentRoutes from './routes/payments.js';

dotenv.config();

const app = express();
app.use(cors());
// ■■■■■■■ 100kb■■■■■■■■ + ■■■■■■■■■■■■■■ 1mb
app.use(express.json({ limit: '1mb' }));

app.get('/api/health', (_req, res) => res.json({ code: 200, message: 'success', data: { status: 'ok' } }));
app.use('/api/auth', authRoutes);
app.use('/api/users', userRoutes);
app.use('/api/bikes', bikeRoutes);
app.use('/api/stations', stationRoutes);
app.use('/api/orders', orderRoutes);
app.use('/api/maintenance', maintenanceRoutes);
app.use('/api/announcements', announcementRoutes);
app.use('/api/dashboard', dashboardRoutes);
app.use('/api/ai', aiRoutes);
app.use('/api/models', modelRoutes);
app.use('/api/payments', paymentRoutes);

app.use((req, res) => res.status(404).json({ code: 404, message: `Not found: ${req.path}` }));
app.use((err, _req, res, _next) => {
  const status = err.status || 500;
  res.status(status).json({ code: status, message: err.message || 'Internal server error' });
});

export default app;
```

源码文件：backend/src/config/db.js

SHA-256：de5bd8166b1542dd7faa600902fff87cff9333dfedf471a3ea0c2cc26f5150f3

```
import pg from 'pg';
import dotenv from 'dotenv';

dotenv.config();

const { Pool, types } = pg;
types.setTypeParser(20, (value) => {
  const number = Number(value);
  return Number.isSafeInteger(number) ? number : value;
});

function bindPlaceholders(sql) {
```

```
  let index = 0;
  let quote = null;
  let lineComment = false;
  let blockComment = false;
  let result = '';

  for (let i = 0; i < sql.length; i += 1) {
    const char = sql[i];
    const next = sql[i + 1];
    if (lineComment) {
      result += char;
      if (char === '\n') lineComment = false;
      continue;
    }
    if (blockComment) {
      result += char;
      if (char === '*' && next === '/') {
        result += next;
        i += 1;
        blockComment = false;
      }
      continue;
    }
    if (quote) {
      result += char;
      if (char === quote && next === quote) {
        result += next;
        i += 1;
      } else if (char === quote && sql[i - 1] !== '\\') {
        quote = null;
      }
      continue;
    }
    if (char === '-' && next === '-') {
      result += `${char}${next}`;
      i += 1;
      lineComment = true;
    } else if (char === '/' && next === '*') {
      result += `${char}${next}`;
      i += 1;
      blockComment = true;
    } else if (char === "'" || char === '"') {
      quote = char;
      result += char;
    } else if (char === '?') {
      result += `$${++index}`;
    } else {
      result += char;
    }
  }
  return result;
}

const poolOptions = process.env.DATABASE_URL
  ? { connectionString: process.env.DATABASE_URL }
  : {
      host: process.env.PGHOST || '127.0.0.1',
      port: Number(process.env.PGPORT || 5432),
      user: process.env.PGUSER || 'student_bike_rental_app',
      password: process.env.PGPASSWORD || undefined,
      database: process.env.PGDATABASE || 'student_bike_rental',
      ssl: process.env.PGSSL === '1' ? { rejectUnauthorized: true } : undefined
    };

if (process.env.NODE_ENV === 'production' && !process.env.PGPASSWORD && !process.env.DATABASE_URL) {
  throw new Error('Set PGPASSWORD or DATABASE_URL for PostgreSQL.');
}

const rawPool = new Pool({ ...poolOptions, max: Number(process.env.PGPOOL_MAX || 10) });

function adapt(client) {
  return {
    async query(sql, params) {
      return client.query(bindPlaceholders(sql), params);
    },
    async execute(sql, params = []) {
      const result = await client.query(bindPlaceholders(sql), params.map((value) => value === undefined ? null : value));
      return [result.rows, { affectedRows: result.rowCount, rowCount: result.rowCount }];
    },
    release: client.release?.bind(client)
  };
}

export const pool = {
  async query(sql, params) {
    return rawPool.query(bindPlaceholders(sql), params);
  },
  async execute(sql, params = []) {
    const result = await rawPool.query(bindPlaceholders(sql), params.map((value) => value === undefined ? null : value));
    return [result.rows, { affectedRows: result.rowCount, rowCount: result.rowCount }];
  },
```

```
  async getConnection() {
    return adapt(await rawPool.connect());
  },
  end() {
    return rawPool.end();
  }
};

export async function tx(work) {
  const conn = await pool.getConnection();
  try {
    await conn.query('BEGIN');
    const result = await work(conn);
    await conn.query('COMMIT');
    return result;
  } catch (error) {
    await conn.query('ROLLBACK').catch(() => {});
    throw error;
  } finally {
    conn.release();
  }
}

export { bindPlaceholders };
```

源码文件：backend/src/middleware/auth.js

SHA-256：901ee85dc54c4610643a4b651d85af2d01a161d029b4ed5f76a64f392fec8743

```
import jwt from 'jsonwebtoken';
import { HttpError } from '../utils/errors.js';

export function auth(required = true) {
  return (req, _res, next) => {
    const header = req.headers.authorization || '';
    const token = header.startsWith('Bearer ') ? header.slice(7) : '';
    if (!token) {
      if (!required) return next();
      return next(new HttpError(401, 'Unauthorized'));
    }
    try {
      req.user = jwt.verify(token, process.env.JWT_SECRET || 'dev_secret');
      return next();
    } catch {
      return next(new HttpError(401, 'Invalid token'));
    }
  };
}
```

源码文件：backend/src/middleware/role.js

SHA-256：ade1eac53481655c8f23c081dd32071e1e38397cc176d24dd1d6f450b2fe8d2f

```
import { HttpError } from '../utils/errors.js';

export function allow(...roles) {
  return (req, _res, next) => {
    if (!req.user || !roles.includes(req.user.role)) {
      return next(new HttpError(403, 'Forbidden'));
    }
    return next();
  };
}
```

源码文件：backend/src/routes/auth.js

SHA-256：d554a7afa534a1b77b5e04d959a2cadded4e8a00c7ee19f105e9e33e9ef3c0cc

```
import { Router } from 'express';
import bcrypt from 'bcryptjs';
import jwt from 'jsonwebtoken';
import { pool, tx } from '../config/db.js';
import { auth } from '../middleware/auth.js';
import { asyncHandler, ok } from '../utils/response.js';
import { HttpError } from '../utils/errors.js';
import { createLimiter, clientIp } from '../utils/rateLimit.js';
import { verifyOidcAccessToken } from '../auth/oidc.js';

const router = Router();
const publicUserFields = 'id, username, name, email, phone, role, student_no, nationality, language, status, created_at';
const MIN_PASSWORD_LENGTH = 6;
```

```
/**
 * ██████**█████**█████ JWT_SECRET ████████
 * ████████████████ auth() ████████████
 * ████████████████——██████ purpose ████████
 */
function resetSecret() {
  return `${process.env.JWT_SECRET || 'dev_secret'}:password-reset`;
}

// ██████████"█████████"██████████████████
// ███ IP ██████████████████
const verifyLimiterByIp = createLimiter({ windowMs: 10 * 60 * 1000, max: 5 });
const verifyLimiterByUser = createLimiter({ windowMs: 10 * 60 * 1000, max: 3 });
const oidcLoginLimiter = createLimiter({ windowMs: 15 * 60 * 1000, max: 30 });
const oidcLinkLimiter = createLimiter({ windowMs: 15 * 60 * 1000, max: 10 });

/** ██████████████████"███████"██"█████"██████████ */
const IDENTITY_MISMATCH = 'Identity information does not match our records';

router.post('/register', asyncHandler(async (req, res) => {
  const { username, password, name, email, phone, student_no, nationality, language = 'en-US', model_id } = req.body;
  if (!['ms-MY', 'en-US', 'vi-VN', 'lo-LA'].includes(language)) throw new HttpError(400, 'Unsupported language');
  if (!username || !password) throw new HttpError(400, 'Username and password are required');
  if (String(username).length < 3) throw new HttpError(400, 'Username must be at least 3 characters');
  if (String(password).length < MIN_PASSWORD_LENGTH) {
    throw new HttpError(400, `Password must be at least ${MIN_PASSWORD_LENGTH} characters`);
  }

  // ████████████████████████████████████████
  const [exists] = await pool.execute('SELECT id FROM users WHERE lower(username) = lower(?)', [username]);
  if (exists.length) throw new HttpError(409, 'Username already exists');

  const hash = await bcrypt.hash(password, 10);
  try {
    await pool.execute(
      `INSERT INTO users (username, password, name, email, phone, role, student_no, nationality, language, status, preferred_model_id)
       VALUES (?, ?, ?, ?, ?, 'student', ?, ?, ?, 'active',
         (SELECT id FROM bike_models WHERE id=? AND status='published'))`,
      [username, hash, name || username, email || null, phone || null, student_no || null, nationality || null, language, model_id || null]
    );
  } catch (err) {
    if (err.code === '23505') throw new HttpError(409, 'Username already exists');
    throw err;
  }
  ok(res, null, 'registered');
}));

router.post('/login', asyncHandler(async (req, res) => {
  const { username, password } = req.body;
  const [rows] = await pool.execute(`SELECT * FROM users WHERE lower(username) = lower(?) AND status = 'active'`, [username]);
  const user = rows[0];
  if (!user || !(await bcrypt.compare(password || '', user.password))) {
    throw new HttpError(401, 'Invalid username or password');
  }
  const token = jwt.sign({ id: user.id, username: user.username, role: user.role }, process.env.JWT_SECRET || 'dev_secret', { expiresIn: '7d
  delete user.password;
  ok(res, { token, user });
}));

router.post('/oidc', asyncHandler(async (req, res) => {
  if (!oidcLoginLimiter(clientIp(req)).allowed) throw new HttpError(429, 'Too many sign-in attempts');
  let identity;
  try { identity = await verifyOidcAccessToken(req.body?.accessToken); }
  catch { throw new HttpError(401, 'Invalid identity token'); }

  const [rows] = await pool.execute("SELECT * FROM users WHERE oidc_subject = ? AND status = 'active'", [identity.subject]);
  const user = rows[0];
  if (!user) throw new HttpError(403, 'No active local account is linked. Sign in locally and link it from your profile first.');
  const token = jwt.sign({ id: user.id, username: user.username, role: user.role }, process.env.JWT_SECRET || 'dev_secret', { expiresIn: '7d
  delete user.password;
  delete user.oidc_subject;
  ok(res, { token, user });
}));

router.post('/oidc/link', auth(), asyncHandler(async (req, res) => {
  if (!oidcLinkLimiter(`${clientIp(req)}:${req.user.id}`).allowed) throw new HttpError(429, 'Too many identity-link attempts');
  let identity;
  try { identity = await verifyOidcAccessToken(req.body?.accessToken); }
  catch { throw new HttpError(401, 'Invalid identity token'); }

  try {
    await tx(async (connection) => {
      const [rows] = await connection.execute("SELECT id, oidc_subject FROM users WHERE id = ? AND status = 'active' FOR UPDATE", [req.user.
      const user = rows[0];
      if (!user) throw new HttpError(403, 'This local account is unavailable');
      if (user.oidc_subject && user.oidc_subject !== identity.subject) throw new HttpError(409, 'This local account is already linked to ano
      const [existing] = await connection.execute('SELECT id FROM users WHERE oidc_subject = ? AND id <> ?', [identity.subject, req.user.id]
      if (existing.length) throw new HttpError(409, 'This identity is already linked to another local account');
      await connection.execute('UPDATE users SET oidc_subject = ? WHERE id = ?', [identity.subject, req.user.id]);
    });
  } catch (error) {
```

```
    if (error.code === '23505') throw new HttpError(409, 'This identity is already linked to another local account');
    throw error;
  }
  ok(res, { linked: true });
}));

router.get('/oidc/status', auth(), asyncHandler(async (req, res) => {
  const [rows] = await pool.execute('SELECT oidc_subject IS NOT NULL AS linked FROM users WHERE id = ?', [req.user.id]);
  if (!rows[0]) throw new HttpError(404, 'User not found');
  ok(res, { linked: rows[0].linked });
}));

router.get('/profile', auth(), asyncHandler(async (req, res) => {
  const [rows] = await pool.execute(`SELECT ${publicUserFields} FROM users WHERE id = ?`, [req.user.id]);
  ok(res, rows[0]);
}));

router.put('/profile', auth(), asyncHandler(async (req, res) => {
  const { name, email, phone, nationality, language } = req.body;
  const [rows] = await pool.execute('SELECT * FROM users WHERE id = ?', [req.user.id]);
  const current = rows[0];
  if (!current) throw new HttpError(404, 'User not found');
  if (language !== undefined && !['ms-MY', 'en-US', 'vi-VN', 'lo-LA'].includes(language)) throw new HttpError(400, 'Unsupported language');

  await pool.execute(
    'UPDATE users SET name=?, email=?, phone=?, nationality=?, language=? WHERE id=?',
    [
      name ?? current.name,
      email ?? current.email,
      phone ?? current.phone,
      nationality ?? current.nationality,
      language ?? current.language,
      req.user.id
    ]
  );
  ok(res, null, 'updated');
}));

/** ■■■■■■■■■■■■■■■■ */
router.put('/password', auth(), asyncHandler(async (req, res) => {
  const { oldPassword, newPassword } = req.body;
  if (!newPassword || String(newPassword).length < MIN_PASSWORD_LENGTH) {
    throw new HttpError(400, `New password must be at least ${MIN_PASSWORD_LENGTH} characters`);
  }
  if (oldPassword === newPassword) throw new HttpError(400, 'New password must differ from the old one');

  const [rows] = await pool.execute('SELECT password FROM users WHERE id=?', [req.user.id]);
  if (!rows[0] || !(await bcrypt.compare(oldPassword || '', rows[0].password))) {
    throw new HttpError(400, 'Old password is incorrect');
  }
  await pool.execute('UPDATE users SET password=? WHERE id=?', [await bcrypt.hash(newPassword, 10), req.user.id]);
  ok(res, null, 'password updated');
}));

/**
 * ■■■■■■■■■■■■■■
 * ■■■■■■ + ■■ + ■■■■■■■■■■■■■■■■■■■■■■■■■
 * ■■■■■■■■■■ SMTP ■■■■■
 *
 * ■■■email / student_no ■■■■■■ NULL■■ `email = ?` ■■■■■■ NULL■
 * ■■■■■■■■■■■■■■■■■■■■■■
 */
router.post('/forgot-password/verify', asyncHandler(async (req, res) => {
  const { username, student_no, email } = req.body;

  const ipCheck = verifyLimiterByIp(clientIp(req));
  if (!ipCheck.allowed) {
    throw new HttpError(429, `Too many attempts. Try again in ${Math.ceil(ipCheck.retryAfterMs / 60000)} minute(s)`);
  }
  const userCheck = verifyLimiterByUser(String(username || '').trim().toLowerCase());
  if (!userCheck.allowed) {
    throw new HttpError(429, `Too many attempts for this account. Try again in ${Math.ceil(userCheck.retryAfterMs / 60000)} minute(s)`);
  }

  if (!username || !student_no || !email) throw new HttpError(400, IDENTITY_MISMATCH);

  const [rows] = await pool.execute(
    `SELECT id, username FROM users
     WHERE lower(username) = lower(?) AND student_no = ? AND email = ? AND status = 'active'`,
    [String(username).trim(), String(student_no).trim(), String(email).trim()]
  );
  const user = rows[0];
  if (!user) throw new HttpError(400, IDENTITY_MISMATCH);

  const resetToken = jwt.sign(
    { id: user.id, username: user.username, purpose: 'password-reset' },
    resetSecret(),
    { expiresIn: '10m' }
  );
  ok(res, { resetToken });
}));
```

```
/** ■■■■■■■■■■■■■■■■■■■■■■■■ */
router.post('/forgot-password/reset', asyncHandler(async (req, res) => {
  const { resetToken, newPassword } = req.body;
  if (!resetToken) throw new HttpError(400, 'Reset token is required');
  if (!newPassword || String(newPassword).length < MIN_PASSWORD_LENGTH) {
    throw new HttpError(400, `New password must be at least ${MIN_PASSWORD_LENGTH} characters`);
  }

  let payload;
  try {
    payload = jwt.verify(resetToken, resetSecret());
  } catch {
    throw new HttpError(400, 'Reset link has expired or is invalid. Please verify your identity again.');
  }
  if (payload.purpose !== 'password-reset') throw new HttpError(400, 'Invalid reset token');

  const [rows] = await pool.execute(`SELECT id FROM users WHERE id=? AND status='active'`, [payload.id]);
  if (!rows.length) throw new HttpError(400, 'Account is not available');

  await pool.execute('UPDATE users SET password=? WHERE id=?', [await bcrypt.hash(newPassword, 10), payload.id]);
  ok(res, null, 'password reset');
}));

export default router;
```

源码文件：backend/src/routes/bikes.js

SHA-256：bc81017c8e03da9e5450464defa224dbc4a0e5a92f54776126a018eb2332884e

```
import { Router } from 'express';
import { pool } from '../config/db.js';
import { auth } from '../middleware/auth.js';
import { allow } from '../middleware/role.js';
import { asyncHandler, ok } from '../utils/response.js';
import { HttpError } from '../utils/errors.js';
import { parsePaging, paginateRows, escapeLike, whereBuilder } from '../utils/paginate.js';

const router = Router();

const STATUSES = ['available', 'rented', 'maintenance', 'disabled'];
const LIST_COLUMNS = 'b.*, s.name_zh AS station_name_zh, s.name_en AS station_name_en';

/**
 * ■■■■■■■■■■■■■■■■■■■■■■ { list, total, page, pageSize }■
 * ■■■`status` ■■■■■■■■■■■■■——■■■■■■■■■■■■■■■■■■■
 */
router.get('/', auth(false), asyncHandler(async (req, res) => {
  const { status, station_id, keyword, type } = req.query;
  const w = whereBuilder();
  w.addIf(status, 'b.status = ?', status);
  w.addIf(station_id, 'b.station_id = ?', station_id);
  w.addIf(type, 'b.type = ?', type);
  w.addIf(keyword, '(b.bike_no ILIKE ? OR b.name ILIKE ?)', `%${escapeLike(keyword)}%`, `%${escapeLike(keyword)}%`);
  const { where, params } = w.build();
  const { page, pageSize } = parsePaging(req.query);

  const result = await paginateRows(pool, {
    select: LIST_COLUMNS,
    from: 'bikes b LEFT JOIN stations s ON s.id = b.station_id',
    where,
    params,
    orderBy: 'b.id DESC',
    page,
    pageSize
  });
  ok(res, result);
}));

router.get('/:id', auth(false), asyncHandler(async (req, res) => {
  const [rows] = await pool.execute('SELECT * FROM bikes WHERE id=?', [req.params.id]);
  ok(res, rows[0] || null);
}));

router.post('/', auth(), allow('admin'), asyncHandler(async (req, res) => {
  const { bike_no, name, type, status = 'available', station_id, hourly_rate = 2, image_url, description } = req.body;
  if (!bike_no) throw new HttpError(400, 'bike_no is required');
  if (!STATUSES.includes(status)) throw new HttpError(400, `Invalid status: ${status}`);

  const [exists] = await pool.execute('SELECT id FROM bikes WHERE bike_no=?', [bike_no]);
  if (exists.length) throw new HttpError(409, 'Bike number already exists');

  await pool.execute(
    'INSERT INTO bikes (bike_no,name,type,status,station_id,hourly_rate,image_url,description) VALUES (?,?,?,?,?,?,?,?)',
    [bike_no, name || bike_no, type || 'standard', status, station_id || null, hourly_rate, image_url || null, description || null]
  );
  ok(res, null, 'created');
}));
router.put('/:id', auth(), allow('admin', 'staff'), asyncHandler(async (req, res) => {
```

```
  const { bike_no, name, type, status, station_id, hourly_rate, image_url, description } = req.body;
  if (status !== undefined && !STATUSES.includes(status)) throw new HttpError(400, `Invalid status: ${status}`);

  const [rows] = await pool.execute('SELECT * FROM bikes WHERE id=?', [req.params.id]);
  const current = rows[0];
  if (!current) throw new HttpError(404, 'Bike not found');

  await pool.execute(
    'UPDATE bikes SET bike_no=?, name=?, type=?, status=?, station_id=?, hourly_rate=?, image_url=?, description=? WHERE id=?',
    [
      bike_no ?? current.bike_no,
      name ?? current.name,
      type ?? current.type,
      status ?? current.status,
      station_id ?? current.station_id,
      hourly_rate ?? current.hourly_rate,
      image_url ?? current.image_url,
      description ?? current.description,
      req.params.id
    ]
  );
  ok(res, null, 'updated');
}));

router.delete('/:id', auth(), allow('admin'), asyncHandler(async (req, res) => {
  // ■■■■rental_orders / maintenance_records ■■■■■■ bikes
  await pool.execute(`UPDATE bikes SET status='disabled' WHERE id=?`, [req.params.id]);
  ok(res, null, 'disabled');
}));

export default router;
```

源码文件：backend/src/routes/orders.js

SHA-256：7660b782288511fec8f5262895892b0373c6484a155dc03d2766a952658db5d3

```
import { Router } from 'express';
import { nanoid } from 'nanoid';
import { pool, tx } from '../config/db.js';
import { auth } from '../middleware/auth.js';
import { allow } from '../middleware/role.js';
import { asyncHandler, ok } from '../utils/response.js';
import { HttpError } from '../utils/errors.js';
import { parsePaging, paginateRows, escapeLike, whereBuilder } from '../utils/paginate.js';
import { getMyOrderPage } from '../services/orderHistory.js';

const router = Router();
router.use(auth());

/** ■■■■■■■■■■■■■■ / ■■ / ■■■■■■■■■■■■■■ */
router.get('/', allow('admin'), asyncHandler(async (req, res) => {
  const { keyword, status, from, to } = req.query;
  const w = whereBuilder();
  w.addIf(status, 'o.status = ?', status);
  w.addIf(keyword, '(o.order_no ILIKE ? OR b.bike_no ILIKE ? OR u.username ILIKE ?)',
    ...Array(3).fill(`%${escapeLike(keyword)}%`));
  // ■■■ >= / < ■■■ BETWEEN■■■ to ■■ 00:00 ■■■■■■■■
  w.addIf(from, 'o.start_time >= ?', from);
  w.addIf(to, "o.start_time < (?::date + INTERVAL '1 day')", to);
  const { where, params } = w.build();
  const { page, pageSize } = parsePaging(req.query);

  const result = await paginateRows(pool, {
    select: `o.*, u.username, u.name AS user_name, b.bike_no, b.name AS bike_name,
             ss.name_zh AS start_station_zh, es.name_zh AS end_station_zh`,
    from: `rental_orders o
           LEFT JOIN users u ON u.id = o.user_id
           LEFT JOIN bikes b ON b.id = o.bike_id
           LEFT JOIN stations ss ON ss.id = o.start_station_id
           LEFT JOIN stations es ON es.id = o.end_station_id`,
    where,
    params,
    orderBy: 'o.id DESC',
    page,
    pageSize
  });
  ok(res, result);
}));

router.get('/my', asyncHandler(async (req, res) => {
  ok(res, await getMyOrderPage(pool, req.user.id, req.query));
}));

router.get('/current', asyncHandler(async (req, res) => {
  const [rows] = await pool.execute(`SELECT * FROM rental_orders WHERE user_id=? AND status='renting' ORDER BY id DESC LIMIT 1`, [req.user.i
  ok(res, rows[0] || null);
}));
router.get('/:id', asyncHandler(async (req, res) => {
```

```
  const [rows] = await pool.execute('SELECT * FROM rental_orders WHERE id=?', [req.params.id]);
  const order = rows[0];
  if (!order) return ok(res, null);
  if (req.user.role !== 'admin' && order.user_id !== req.user.id) throw new HttpError(403, 'Forbidden');
  ok(res, order);
}));

router.post('/rent', allow('student', 'admin'), asyncHandler(async (req, res) => {
  const { bike_id, start_station_id } = req.body;
  let order;
  try {
    order = await tx(async (conn) => {
      await conn.execute('SELECT id FROM users WHERE id=? FOR UPDATE', [req.user.id]);
      const [active] = await conn.execute(`SELECT id FROM rental_orders WHERE user_id=? AND status='renting'`, [req.user.id]);
      if (active.length) throw new HttpError(400, 'You already have an active rental');
      const [bikes] = await conn.execute(`SELECT * FROM bikes WHERE id=? FOR UPDATE`, [bike_id]);
      const bike = bikes[0];
      if (!bike || bike.status !== 'available') throw new HttpError(400, 'Bike is not available');
      await conn.execute(`UPDATE bikes SET status='rented' WHERE id=?`, [bike_id]);
      const orderNo = `RO${Date.now()}${nanoid(6).toUpperCase()}`;
      await conn.execute(
        `INSERT INTO rental_orders (order_no,user_id,bike_id,start_time,hourly_rate,status,start_station_id)
         VALUES (?, ?, ?, NOW(), ?, 'renting', ?)`,
        [orderNo, req.user.id, bike_id, bike.hourly_rate, start_station_id || bike.station_id]
      );
      const [rows] = await conn.execute('SELECT * FROM rental_orders WHERE order_no=?', [orderNo]);
      return rows[0];
    });
  } catch (error) {
    if (error.code === '23505' && error.constraint === 'uniq_rental_orders_active_user') {
      throw new HttpError(400, 'You already have an active rental');
    }
    throw error;
  }
  ok(res, order, 'rented');
}));

router.put('/:id/return', allow('student', 'admin'), asyncHandler(async (req, res) => {
  const { end_station_id } = req.body;
  const result = await tx(async (conn) => {
    const [orders] = await conn.execute(`SELECT * FROM rental_orders WHERE id=? FOR UPDATE`, [req.params.id]);
    const order = orders[0];
    if (!order || order.status !== 'renting') throw new HttpError(400, 'Order is not active');
    if (req.user.role !== 'admin' && order.user_id !== req.user.id) throw new HttpError(403, 'Forbidden');
    const minutesUsed = Math.ceil((Date.now() - new Date(order.start_time).getTime()) / 60000);
    const hours = Math.max(1, Math.ceil(minutesUsed / 60));
    const total = Number((hours * Number(order.hourly_rate)).toFixed(2));
    await conn.execute(
      `UPDATE rental_orders SET end_time=NOW(), duration_hours=?, total_amount=?, status='completed', end_station_id=? WHERE id=?`,
      [hours, total, end_station_id || order.start_station_id, order.id]
    );
    await conn.execute(`UPDATE bikes SET status='available', station_id=? WHERE id=?`, [end_station_id || order.start_station_id, order.bike_
    const [rows] = await conn.execute('SELECT * FROM rental_orders WHERE id=?', [order.id]);
    return rows[0];
  });
  ok(res, result, 'returned');
}));

export default router;
```

源码文件：backend/src/routes/stations.js

SHA-256：d7d3283f18bcfab749af619ae9d56dd376a88f9837175aae0aaaa431221cd209

```
import { Router } from 'express';
import { pool } from '../config/db.js';
import { auth } from '../middleware/auth.js';
import { allow } from '../middleware/role.js';
import { asyncHandler, ok } from '../utils/response.js';
import { HttpError } from '../utils/errors.js';
import { parsePaging, paginateRows, escapeLike, whereBuilder } from '../utils/paginate.js';

const router = Router();

/**
 * ■■■■■■■■■■■■■■■■■ '' ■■■■■■■■■■undefined■■
 * ■■■■■■■■ NULL■■■■■■■■■■■■■■■■■
 */
const nullableNum = (v) => (v === '' || v === undefined ? null : v);

router.get('/', auth(false), asyncHandler(async (req, res) => {
  const { keyword } = req.query;
  const w = whereBuilder();
  w.addIf(keyword, '(s.name_zh ILIKE ? OR s.name_en ILIKE ? OR s.address_zh ILIKE ? OR s.address_en ILIKE ?)',
    ...Array(4).fill(`%${escapeLike(keyword)}%`));
  const { where, params } = w.build();
  const { page, pageSize } = parsePaging(req.query);
  // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

```
  const result = await paginateRows(pool, {
    select: `s.*,
      (SELECT COUNT(*) FROM bikes b WHERE b.station_id = s.id AND b.status = 'available') AS available_count,
      (SELECT COUNT(*) FROM bikes b WHERE b.station_id = s.id) AS bike_count`,
    from: 'stations s',
    where,
    params,
    orderBy: 's.id DESC',
    page,
    pageSize
  });
  ok(res, result);
}));

router.get('/:id', auth(false), asyncHandler(async (req, res) => {
  const [rows] = await pool.execute('SELECT * FROM stations WHERE id=?', [req.params.id]);
  ok(res, rows[0] || null);
}));

router.post('/', auth(), allow('admin'), asyncHandler(async (req, res) => {
  const { name_zh, name_en, address_zh, address_en, latitude, longitude, capacity } = req.body;
  if (!name_zh || !name_en) throw new HttpError(400, 'name_zh and name_en are required');
  await pool.execute(
    'INSERT INTO stations (name_zh,name_en,address_zh,address_en,latitude,longitude,capacity) VALUES (?,?,?,?,?,?,?)',
    [name_zh, name_en, address_zh || null, address_en || null,
      nullableNum(latitude), nullableNum(longitude), capacity || 0]
  );
  ok(res, null, 'created');
}));

router.put('/:id', auth(), allow('admin'), asyncHandler(async (req, res) => {
  const { name_zh, name_en, address_zh, address_en, latitude, longitude, capacity } = req.body;
  const [rows] = await pool.execute('SELECT * FROM stations WHERE id=?', [req.params.id]);
  const current = rows[0];
  if (!current) throw new HttpError(404, 'Station not found');

  await pool.execute(
    'UPDATE stations SET name_zh=?, name_en=?, address_zh=?, address_en=?, latitude=?, longitude=?, capacity=? WHERE id=?',
    [
      name_zh ?? current.name_zh,
      name_en ?? current.name_en,
      address_zh ?? current.address_zh,
      address_en ?? current.address_en,
      // ■■■■ -> ■■ NULL■■■■■■■ -> ■■■■
      nullableNum(latitude) ?? current.latitude,
      nullableNum(longitude) ?? current.longitude,
      capacity ?? current.capacity,
      req.params.id
    ]
  );
  ok(res, null, 'updated');
}));

router.delete('/:id', auth(), allow('admin'), asyncHandler(async (req, res) => {
  // bikes.station_id ■■■ ON DELETE SET NULL■■■■■■■■■"■■■"■
  // ■■■■■■■■■■■■■■■■■■■■■■■
  const [[{ count }]] = await pool.execute('SELECT COUNT(*) AS count FROM bikes WHERE station_id=?', [req.params.id]);
  if (Number(count) > 0) throw new HttpError(409, `Station still has ${count} bike(s). Move them first.`);
  const [[{ history_count }]] = await pool.execute(
    'SELECT COUNT(*) AS history_count FROM rental_orders WHERE start_station_id=? OR end_station_id=?',
    [req.params.id, req.params.id]
  );
  if (Number(history_count) > 0) throw new HttpError(409, 'Station is referenced by rental history.');
  try {
    await pool.execute('DELETE FROM stations WHERE id=?', [req.params.id]);
  } catch (error) {
    if (error.code === '23503') {
      throw new HttpError(409, 'Station is referenced by rental history.');
    }
    throw error;
  }
  ok(res, null, 'deleted');
}));

export default router;
```

源码文件：backend/src/routes/ai.js

SHA-256：bd3c09258622d69d65d2c86ec3b15be254b1d24fa9f0648a72991dc2770c1591

```
import express from 'express';
import { auth } from '../middleware/auth.js';
import { allow } from '../middleware/role.js';
import { ok, asyncHandler } from '../utils/response.js';
import { HttpError } from '../utils/errors.js';
import { getCatalog, invalidateCatalog } from '../ai/catalog.js';
import { aiConfig, parseModelId } from '../ai/config.js';
import { runAgent } from '../ai/agent.js';
```

```
import { createNdjsonStream } from '../ai/stream.js';
import { acquireSlot, checkUserQuota, gateStatus } from '../ai/limits.js';
import { describeCapabilities } from '../ai/prompt.js';
import { gpuSnapshot } from '../ai/gpu.js';
import * as ollama from '../ai/provider/ollama.js';

const router = express.Router();

/** ■■■■■■■■■ —— ■■■■■■ num_ctx■■■■■■■■■■■■■■■■■■ */
const MAX_HISTORY = 10;
const MAX_CONTENT_CHARS = 2000;

/**
 * GET /api/ai/models
 * ■■■■■■■■■■■■■■■■■■■■■■■■■**■■■■■**■
 */
router.get('/models', auth(false), asyncHandler(async (req, res) => {
  const catalog = await getCatalog({ refresh: req.query.refresh === '1' });
  return ok(res, {
    groups: catalog.groups,
    defaultModel: catalog.defaultModel,
    capabilities: describeCapabilities(req.user?.role, req.query.lang),
    // ■■■■■■■■■■■■"■■■■■"■■
    gate: gateStatus()
  });
}));

/**
 * POST /api/ai/chat
 * NDJSON ■■■■■■
 *
 * ■■ ■■ handler ■**■■■ next(err)**■res.flushHeaders() ■■■■■■
 * app.js ■■■■■■■■■■■■■■■■■■■■■■ERR_HTTP_HEADERS_SENT■
 * ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
 * ■■■■ {type:'error'} ■■■■■■■■■■■■■■■ end()■
 */
router.post('/chat', auth(), asyncHandler(async (req, res) => {
  const { model, messages, lang } = req.body || {};

  if (!model) throw new HttpError(400, '■■ model ■■');
  if (!Array.isArray(messages) || !messages.length) throw new HttpError(400, 'messages ■■■■');

  // ■■■■■■■■■■■■■■■■■■■■■■■■■■
  const catalog = await getCatalog();
  if (!catalog.models.some((m) => m.id === model)) {
    throw new HttpError(400, `■■■■■■■${model}`);
  }

  // ■■■■■■■■■■■■ 429 ■■■■■■■■ JSON ■■■■■■■■■
  checkUserQuota(req.user.id);

  const history = sanitizeHistory(messages);
  if (!history.length) throw new HttpError(400, 'messages ■■■■■■■■');

  const stream = createNdjsonStream(res);
  const controller = new AbortController();
  stream.onClientClose(() => controller.abort());

  let release = null;
  let lastError = null;

  try {
    // ■■■■■■■■■■■■ 503■■■■■■■■■■■■■■■■■■■
    try {
      release = await acquireSlot();
    } catch (e) {
      stream.write({ type: 'error', message: e.message, code: e.status || 503 });
      stream.close();
      return;
    }

    stream.write({ type: 'start', model, lang: lang || 'zh-CN' });

    for await (const ev of runAgent({
      modelId: model,
      history,
      user: req.user,
      lang: lang || 'zh-CN',
      signal: controller.signal
    })) {
      if (stream.closed) break; // ■■■■■■■/■■■■■■■■■■■■
      stream.write(ev);
    }
  } catch (e) {
    lastError = e;
    if (e.name !== 'AbortError') {
      console.error('[ai] ■■■■■■', e.message);
      stream.write({
        type: 'error',
        message: friendlyError(e, model),
        code: e.status || 500
```

```
      });
    }
  } finally {
    release?.();
    stream.close();
    if (lastError && lastError.name !== 'AbortError') {
      // ■■■■■■■■■■■■■■■■■■■■■■■■
      console.warn('[ai] ■■■■■■■■■%s', lastError.message);
    }
  }
}));

/** ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
function friendlyError(e, modelId) {
  const { provider } = parseModelId(modelId || '');
  const msg = String(e.message || '');
  if (/ECONNREFUSED|■■■ Ollama/.test(msg)) {
    return '■■■■■■■ Ollama ■■■■■■■ Ollama ■■■■■■';
  }
  if (e.status === 401 || /401/.test(msg)) {
    return '■■■■■■■■■■■■■■■ API Key ■■■';
  }
  if (e.status === 429 || /429/.test(msg)) {
    return '■■■■■■■■■■■■■■■■';
  }
  if (/context length|too long|num_ctx/i.test(msg)) {
    return '■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■';
  }
  if (provider === 'ollama' && /not found|no such model/i.test(msg)) {
    return '■■■■■■■■■■■■■■■■■■■■■■■■';
  }
  return msg.slice(0, 300) || '■■■■■■■■■■■■■■';
}

/**
 * ■■■■■■■■■■■■
 * ■■■ user/assistant ■■■■■——**■■■■■■ tool / system ■■■■■■■**■
 * ■■■■■■■■■■■■■■ system ■■■■■■■■■■■■■■■■
 */
function sanitizeHistory(messages) {
  return messages
    .filter((m) => m && (m.role === 'user' || m.role === 'assistant'))
    .filter((m) => typeof m.content === 'string' && m.content.trim())
    .slice(-MAX_HISTORY)
    .map((m) => ({
      role: m.role,
      content: m.content.slice(0, MAX_CONTENT_CHARS)
    }));
}

/**
 * GET /api/ai/status — ■■■■■■■■■/■■■■■
 * ■■ 3090 ■■■■■■■■■■■■■■■■■■■■■■■■■■
 */
router.get('/status', auth(), allow('admin'), asyncHandler(async (_req, res) => {
  let running = [];
  let error = '';
  try {
    running = await ollama.listRunning();
  } catch (e) {
    error = e.message;
  }
  // ■■■■■■ Ollama ■■■■■■■■Ollama ■■■■■■■■■■■■■■ GPU
  const gpu = await gpuSnapshot();
  return ok(res, {
    gate: gateStatus(),
    numCtx: aiConfig.numCtx,
    keepAlive: aiConfig.keepAlive,
    running,
    error,
    gpu
  });
}));

/**
 * POST /api/ai/models/:name/load — ■■■■■■■■■■■■■■■■■■■■■■■
 * CORS ■ model ■■■■■■qwen3:14b■■■■■■Express ■■■■■■■■■■
 */
router.post('/models/:name/load', auth(), allow('admin'), asyncHandler(async (req, res) => {
  const name = req.params.name;
  await ollama.loadModel(name);
  invalidateCatalog();
  return ok(res, { model: name, loaded: true });
}));

router.post('/models/:name/unload', auth(), allow('admin'), asyncHandler(async (req, res) => {
  const name = req.params.name;
  await ollama.unloadModel(name);
  invalidateCatalog();
  return ok(res, { model: name, loaded: false });
}));
```

```
export default router;
```

源码文件：backend/src/ai/agent.js

SHA-256：74669cf44672ef20e95c25abaed8872e90ef71791f818594ca456ee028219d7d

```
/**
 * ■■ tool-calling ■■■**provider ■■** —— ■■ canonical ■■/■■■
 *
 * ■■■■■■■
 *   provider ■ → ■■ text ■■ tool_call → ■■■■■■■ → ■■■■■ role:'tool' ■■ → ■■■■■
 *   ■■■■■■■■■■■■
 *
 * ■■ ■■■■■■■■■■■■
```

```
      if (ev.type === 'text') {
        roundText += ev.text;
        yield ev;
      } else if (ev.type === 'thinking') {
        yield ev;
      } else if (ev.type === 'tool_call') {
        calls.push(ev);
      } else if (ev.type === 'done') {
        doneReason = ev.reason;
        if (ev.usage) yield { type: 'usage', usage: ev.usage, round: totalRounds };
      } else if (ev.type === 'error') {
        yield ev;
      }
    }

    // ████████ 4 ████████████████████████████ ── ██████
    if (!calls.length && looksLikeRawToolCall(roundText)) {
      console.warn('[ai] ████"█████"████████████model=%s', modelId);
      // ████████████████████████████████"██████████"
      yield { type: 'reset' };
      yield {
        type: 'text',
        text: lang === 'en-US'
          ? 'Sorry, I got a bit tangled up there. Could you ask that again?'
          : '████████████████████'
      };
      yield { type: 'done', reason: 'degenerate', rounds: totalRounds };
      return;
    }

    if (!calls.length) {
      yield { type: 'done', reason: doneReason, rounds: totalRounds };
      return;
    }

    // ██ ██████ ██
    // ████████ 3 ██assistant ████████████████ tool ██
    const assistantMsg = { role: 'assistant', content: roundText, toolCalls: calls };
    const toolMsgs = [];

    for (const call of calls) {
      const key = `${call.name}:${JSON.stringify(call.arguments || {})}`;

      let result;
      if (seenCalls.has(key)) {
        // ████████ 2 ████████████████████████████
        result = seenCalls.get(key);
      } else {
        yield { type: 'tool_call', id: call.id, name: call.name, arguments: call.arguments };
        result = await executeTool(call.name, call.arguments, ctx);
        seenCalls.set(key, result);
      }

      const payload = result.ok
        ? { ok: true, data: result.data }
        : { ok: false, error: result.error };

      yield {
        type: 'tool_result',
        id: call.id,
        name: call.name,
        ok: result.ok,
        // ██████████████"███████"████████████
        preview: result.ok ? (result.summary || '') : result.error
      };

      toolMsgs.push({
        role: 'tool',
        toolCallId: call.id,
        name: call.name,
        content: truncateForModel(JSON.stringify(payload))
      });
    }

    // ████████ 3 ████████████████████████
    if (toolMsgs.length === calls.length && toolMsgs.length > 0) {
      convo.push(assistantMsg, ...toolMsgs);
    } else {
      console.warn('[ai] ████████████████calls=%d msgs=%d', calls.length, toolMsgs.length);
      yield { type: 'done', reason: 'pairing_failed', rounds: totalRounds };
      return;
    }
  }

  // ████████████████████████████████████████████████
  yield {
    type: 'notice',
    code: 'max_rounds',
    message: lang === 'en-US' ? 'I have reached my lookup limit.' : '██████████'
  };
  yield { type: 'done', reason: 'max_rounds', rounds: totalRounds };
```

```
}
```

源码文件：backend/src/ai/provider/base.js

SHA-256：3a2644f3af0af4dec2c154500c25de4d8780221c7bef2c5565eedf160fd2e4d4

```
/**
 * provider ■■■■■canonical ■■ / ■■■■■ + ■■■■■■■■■■■■■■
 *
 * ■■■■■■**agent.js ■■ canonical ■■■■■■■■■■■■■■■■**■
 * ■■■■■■■■■"Ollama ■ arguments ■■■■■OpenAI ■■ JSON ■■■■"■■■■■■
 * ■■■■■■■■■■■■■■■■■■■■■■■■■
 *
 * ■■ canonical ■■ ■■■■■■■■■■■■■■■■■■■
```

```
    // ■■■■■■■ ```json ... ``` ■■■■■
    const fenced = text.match(/```(?:json)?\s*([\s\S]*?)```/i);
    if (fenced) {
      try {
        const parsed = JSON.parse(fenced[1].trim());
        return parsed && typeof parsed === 'object' ? parsed : {};
      } catch { /* ■■■■ */ }
    }
    return {};
  }
}

/**
 * ■■ content ■■  thinking...<■end■of■thinking■>■
 * qwen3 ■■■■ think:false ■■■■■■■■■■■ content■
 * ■■■■■**■■■■ TTS**——■■■■■■■■■■■■■■■■
 */
export function stripThinking(text) {
  if (!text) return '';
  const value = String(text);
  // Some Ollama builds emit the reasoning body without its opening tag and
  // only append `</think>` at the end. In that case the whole prefix is still
  // internal reasoning and must not reach the client.
  const close = value.match(/<\/?think(?:ing)?>|<■end■of■thinking■>/i);
  if (close && /^<\//.test(close[0])) return value.slice(close.index + close[0].length);
  // ■■ ■■**■■■■ trim()**■■■■■■■■■■**■■■■**■■■■■■■■■
  // ■■■■■■■■ ["This", " system", " lets", " students"]■■■■■■■■■■■■
  // ■■■■■■■■trim ■■■■■■■■ "Thissystemletsstudents" —— ■■■■■■■■■
  // ■■■■■■■■■■■■■■■■■■■■■ TTS ■■■■■
  // ■■■■■"■■■■■■■"■■■■■■■ trim■
  return value
    .replace(/<think(?:ing)?>[\s\S]*?<\/think(?:ing)?>/gi, '')
    .replace(/<think(?:ing)?>[\s\S]*$/i, '')
    .replace(/<■end■of■thinking■>/gi, '');
}

/**
 * qwen3 ■"■■■■■"■■■■■■
 * ■■■■■■■■■"■■■■■■ available_bikes"■■■■■■■ tool_calls■
 * ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
 */
export function looksLikeRawToolCall(text) {
  if (!text) return false;
  return /<tool_call>|<\/tool_call>|"name"\s*:\s*"[a-z_]+"\s*,\s*"arguments"/i.test(text);
}

/** ■■■■ HTTP ■■■■■■■ status ■ ProviderError■■■■■■■■■ 401/429 */
export async function toProviderError(response, label) {
  let detail = '';
  try {
    detail = (await response.text()).slice(0, 500);
  } catch { /* ■■■■■■■ */ }
  const status = response.status;
  return new ProviderError(
    `${label} ■■ ${status}${detail ? `■${detail}` : ''}`,
    { status, retryable: status === 429 || status >= 500 }
  );
}
```

源码文件：backend/src/ai/provider/ollama.js

SHA-256：720ffbf6b961d8c34e5dccde65425b95bf2a43a9d9f392adb0094b454ecfcc0b

```
/**
 * Ollama ■■ /api/chat ■■■■NDJSON ■■■
 *
 * ■■■■■■■■■■■■■■■ OpenAI ■■■■■
 *   1) ■■■■ `done_reason`■`eval_count` ■■■■■■■■■■"■■■■■■■■■"
 *   2) `think: false` ■ `keep_alive` ■■■■■■■■■■■■■■
 *   3) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
 *
 * ■■ qwen3 + ■■■■■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
 *   1. **■■■■■■ think:false**■■■■■■■■■■■■■ token ■■■■ token ■■■■■■
 *      ■■■■■■ tool_call ■■■■■■■ token ■■■■
 *   2. **"■■■■■"■■**■■■■■■■■■"■■■■■■■■■"■■■■■ tool_calls■
 *      ■■■■■■■■■■■■■■■ → ■ looksLikeRawToolCall■
 *   3. **■■■■■**■assistant(tool_calls) ■■■■■■ role:'tool' ■■■■■■
 *      ■■■■■■■■■■■■ agent.js ■■■■■■■■■■■■■■■■■
 */
import { aiConfig } from '../config.js';
import {
  ProviderError, toProviderError, normalizeToolDef, parseToolArguments, stripThinking
} from './base.js';

/** canonical ■■ → Ollama ■■ */
export function toOllamaMessages(messages) {
  return messages.map((m) => {
    if (m.role === 'tool') {
```

```
      return { role: 'tool', content: m.content ?? '', ...(m.name ? { tool_name: m.name } : {}) };
    }
    if (m.role === 'assistant' && m.toolCalls?.length) {
      return {
        role: 'assistant',
        content: m.content || '',
        tool_calls: m.toolCalls.map((tc) => ({
          function: { name: tc.name, arguments: tc.arguments || {} }
        }))
      };
    }
    return { role: m.role, content: m.content ?? '' };
  });
}

/**
 * ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
 */
function stalledError(model, ms) {
  return new ProviderError(
    `■■ ${model} ■■ ${Math.round(ms / 1000)} ■■■■■■■` +
    '■■■■■■■■■■■■■■■■■■■■■■■■■ GPU ■■■■■■■■■■■■■■■■■■■■',
    { retryable: true }
  );
}

/**
 * ■■■■■■■ read()——■■■**■■■■■■**■■■■■■■■
 *
 * ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
 * ■■■■■■■■■■■■■■■■■■■■"■■■■■■"■"■■■■■"■
 */
function readWithIdleTimeout(reader, ms, model) {
  let timer;
  return Promise.race([
    reader.read(),
    new Promise((_, reject) => {
      timer = setTimeout(() => reject(stalledError(model, ms)), ms);
    })
  ]).finally(() => clearTimeout(timer));
}

/**
 * ■■■■■ fetch■■■■■**■■■■■■■■**■■■■
 *
 * ■■ ■■■■■■■■■■■■Ollama ■■■■■■■■■■**■■■■■■■■**■
 * ■■"■■■■■■■"■■■■■■■■ `await fetch(...)` ■■■■■■■■■■■■——
 * ■■■■■■■■■■■■■■**■■■■**■■■■■■■■■■■■85 ■■■■■■■ error ■■■■■■■■
 *
 * ■■■■■■■ AbortController■**■■**■■■■■■■■■ signal ■■■■■■■
 * ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
 * ■■■■■■ clearTimeout■■■■■■ controller ■■■■■ abort■
 */
async function fetchWithHeaderTimeout(url, init, { model, signal, ms }) {
  const header = new AbortController();
  const timer = setTimeout(() => header.abort(), ms);
  try {
    return await fetch(url, {
      ...init,
      signal: signal ? AbortSignal.any([signal, header.signal]) : header.signal
    });
  } catch (e) {
    // ■■■■■■■■■■■■■■/■■■■■■■■■■■■■■■■■■■■■
    if (signal?.aborted) throw e;
    if (header.signal.aborted) throw stalledError(model, ms);
    throw e;
  } finally {
    clearTimeout(timer);
  }
}

/** canonical ■■■■■ → Ollama tools */
export function toOllamaTools(tools) {
  return (tools || []).map((t) => {
    const def = normalizeToolDef(t);
    return { type: 'function', function: def };
  });
}

/**
 * ■■■■■■■■■■■ async generator■■■■■ canonical ■■■■
 *
 * @param {object} opts
 * @param {string} opts.model      ■■■■■■■ 'qwen3:14b'■**■■** provider ■■■■
 * @param {Array}  opts.messages   canonical ■■■■
 * @param {Array}  [opts.tools]    canonical ■■■■■■
 * @param {AbortSignal} [opts.signal]
 */
export async function* streamChat({ model, messages, tools, signal }) {
  const options = { num_predict: aiConfig.maxTokens };
  // ■■ ■■■■■■■■■■■ num_ctx■■■■■■■■■■■■**■■■■■**■
```

```
// ■■■■■■■■■■■■Ollama ■■■■■■■■■■■■■■■■■■■■■■■■■■
// ■■■■"■■■■■■■■■■■■ token ■■■■■"■■■■ config.js ■ numCtx ■■■■■
if (aiConfig.numCtx > 0) options.num_ctx = aiConfig.numCtx;

const body = {
  model,
  messages: toOllamaMessages(messages),
  stream: true,
  // ■■■■■■■■ 1 ■■■■■■■■■■■■■■■ token ■■■■■■
  think: false,
  keep_alive: aiConfig.keepAlive,
  options
};
if (tools?.length) body.tools = toOllamaTools(tools);

let response;
try {
  response = await fetchWithHeaderTimeout(`${aiConfig.ollamaBaseUrl}/api/chat`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(body)
  }, { model, signal, ms: aiConfig.requestTimeoutMs });
} catch (e) {
  if (e.name === 'AbortError') throw e;
  // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  if (e instanceof ProviderError) throw e;
  // ■■■■ Ollama ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ fetch failed
  throw new ProviderError(`■■■■ Ollama■${aiConfig.ollamaBaseUrl}■■${e.message}`, { retryable: true });
}

if (!response.ok) throw await toProviderError(response, 'Ollama');

const reader = response.body.getReader();
// ■■ ■■■■ {stream:true}■■■■■■■■■■■■■■■■■
const decoder = new TextDecoder('utf-8');
let buffer = '';
let sawToolCall = false;

try {
  while (true) {
    const { done, value } = await readWithIdleTimeout(reader, aiConfig.idleTimeoutMs, model);
    if (done) break;
    buffer += decoder.decode(value, { stream: true });

    // NDJSON■■■■■■ JSON■■■■■■■■■■■■■■■ buffer ■■■■■■ chunk
    let nl;
    while ((nl = buffer.indexOf('\n')) >= 0) {
      const line = buffer.slice(0, nl).trim();
      buffer = buffer.slice(nl + 1);
      if (!line) continue;

      let chunk;
      try {
        chunk = JSON.parse(line);
      } catch {
        continue; // ■■/■■■■■■■■■■■■■■■■
      }

      if (chunk.error) throw new ProviderError(`Ollama ■■■■${chunk.error}`);

      const msg = chunk.message || {};
      if (msg.thinking) yield { type: 'thinking', text: msg.thinking };

      const text = stripThinking(msg.content || '');
      if (text) {
        // ■■■■■■■■ 2 ■■■■"■■■■■"■■■■■■■■■■
        if (!msg.tool_calls?.length) {
          yield { type: 'text', text };
        }
      }

      if (msg.tool_calls?.length) {
        sawToolCall = true;
        for (const tc of msg.tool_calls) {
          const fn = tc.function || {};
          yield {
            type: 'tool_call',
            // Ollama ■■■■■ id■■■■■■■■■■■agent ■■■■■■■■■
            id: tc.id || `call_${fn.name}_${Math.random().toString(36).slice(2, 8)}`,
            name: fn.name,
            // ■ base.js■Ollama ■■■■■■■■■■■■■■■■■■■■■■■■■
            arguments: parseToolArguments(fn.arguments)
          };
        }
      }

      if (chunk.done) {
        yield {
          type: 'done',
          reason: chunk.done_reason || (sawToolCall ? 'tool_calls' : 'stop'),
          usage: {
```

```
              promptTokens: chunk.prompt_eval_count ?? null,
              completionTokens: chunk.eval_count ?? null
            }
          };
          return;
        }
      }
    }
    // ■■■■■■■■■■ done ── ■■■■■■■■■
    yield { type: 'done', reason: 'eof', usage: {} };
  } finally {
    // ■■ return / ■■■■■■■■■■■■■■■ Ollama ■■■■■■■■■■
    try { await reader.cancel(); } catch { /* ■■■■ */ }
  }
}

/** ■■■■■■■■■■■ */
export async function listModels({ signal } = {}) {
  const response = await fetch(`${aiConfig.ollamaBaseUrl}/api/tags`, { signal });
  if (!response.ok) throw await toProviderError(response, 'Ollama /api/tags');
  const data = await response.json();
  return (data.models || []).map((m) => ({
    name: m.name,
    sizeBytes: m.size ?? null,
    parameterSize: m.details?.parameter_size || '',
    quantization: m.details?.quantization_level || '',
    family: m.details?.family || '',
    modifiedAt: m.modified_at || null
  }));
}

/** ■■■■■■■■■■■■■■■■■■■ */
export async function listRunning({ signal } = {}) {
  const response = await fetch(`${aiConfig.ollamaBaseUrl}/api/ps`, { signal });
  if (!response.ok) throw await toProviderError(response, 'Ollama /api/ps');
  const data = await response.json();
  return (data.models || []).map((m) => ({
    name: m.name,
    sizeVram: m.size_vram ?? null,
    sizeBytes: m.size ?? null,
    contextLength: m.context_length ?? null,
    expiresAt: m.expires_at || null
  }));
}

/**
 * ■■■■■■■■■■■■■■■■■■■■"■■"■■■■■■
 * ■■ prompt ■■■■■■ Ollama ■■■■■■■■■■■■■■■■■■
 */
export async function loadModel(model, { signal } = {}) {
  const response = await fetch(`${aiConfig.ollamaBaseUrl}/api/generate`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ model, prompt: '', keep_alive: aiConfig.keepAlive, stream: false }),
    signal
  });
  if (!response.ok) throw await toProviderError(response, 'Ollama /api/generate');
  return true;
}

/** ■■■■■■keep_alive:0 ■ Ollama ■■■■■■■■■■■■ */
export async function unloadModel(model, { signal } = {}) {
  const response = await fetch(`${aiConfig.ollamaBaseUrl}/api/generate`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ model, keep_alive: 0, stream: false }),
    signal
  });
  if (!response.ok) throw await toProviderError(response, 'Ollama /api/generate');
  return true;
}
```

源码文件：backend/src/ai/tools/index.js

SHA-256：2d4bc971f83fb36602f0d3dbf888dbe1aae837cba60fa4e5d865d0485d4d1213

```
/**
 * ■■■■ + ■■ + ■■■■■■■■■■■■■■■■■■■■"■■■■■■■■■■■■"■
 *
 * ■■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
 *   1. **■■■■■**■toolsForRole() ■■■■■■■■■■■■■■■■■■■■■**■■■■■■**
 *      admin_* ■■■■■■■■■■■■■■■■■■■■■■■■■■
 *   2. **■■■■■**■executeTool() ■■■■■ roles■■■■■■■■■■■──
 *      ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ warn ■■■
 *      ■■■■■■■"■■■■■■■■"■■■■■
 *   3. **■■■■■**■■■"■■"■■■**■■■■■■ user_id ■■**■■■■■■■■■■
 *      ctx.user.id ■■■■■■■■■■■■■■■■■■■■■■■■
 *   4. **■■■■■**■SQL ■■■ SELECT **■■■■■**■■■ SELECT *■
```

```
 *      ████████ SELECT *██████████users.phone / email / student_no
 *      ████ JOIN ██████████████████
 *
 * ████████████████**██**████████████████████
 * "███ admin_revenue_stats ████████"——system ███████████
 * ████████ {ok, data} ██████"██"███████████
 */
import { pool } from '../../config/db.js';

/** ███████████ */
const COMMON = ['get_my_current_order', 'get_my_orders', 'list_stations', 'get_bike_info', 'estimate_rental_cost', 'list_announcements'];
/** █████ + ███ */
const STAFF = ['admin_bike_status_summary'];
/** ████ */
const ADMIN = ['admin_revenue_stats', 'admin_user_orders'];

/**
 * ██████████
 *
 * ██████████14B ■ 8 ██████████████████
 * ████████████ get_bike_info / estimate_rental_cost██████████
 * ████████████████████████ 7 ■■
 */
export function toolsForRole(role) {
  let names = [...COMMON];
  if (role === 'staff') {
    names = names.filter((n) => n !== 'estimate_rental_cost').concat(STAFF);
  } else if (role === 'admin') {
    names = names
      .filter((n) => n !== 'estimate_rental_cost' && n !== 'get_bike_info')
      .concat(STAFF, ADMIN);
  }
  // ■■ ■■■**███████ key ■■**■TOOLS ██████████ name■
  // ███████ `map((n) => TOOLS[n])`██████████████**████████**■
  // ████████████ → ██████"█████  ███" → ███████
  // "███████████████"████████████████████████
  return names.map((n) => ({ ...TOOLS[n], name: n }));
}

const MAX_LIMIT = 20;
const clampLimit = (v, dflt = 5) => {
  const n = Number.parseInt(v, 10);
  if (!Number.isFinite(n)) return dflt;
  return Math.min(Math.max(n, 1), MAX_LIMIT);
};

/** ■ orders.js ████████████████ —— ██████████████████ */
export function calcFare(hourlyRate, hours) {
  const h = Math.max(1, Math.ceil(Number(hours) || 0));
  return { hours: h, total: Number((h * Number(hourlyRate || 0)).toFixed(2)) };
}

const TOOLS = {
  // ████████████████████████████████ ■■ ████████████████████████████████
  get_my_current_order: {
    roles: ['student', 'admin', 'staff'],
    description: '■■"■"██████████████████ renting██████"█████████""██████████"████████████████',
    parameters: { type: 'object', properties: {} },
    async run(_args, ctx) {
      const [rows] = await pool.execute(
        `SELECT o.order_no, o.start_time, o.hourly_rate, o.status,
                b.bike_no, b.name AS bike_name, b.type AS bike_type
         FROM rental_orders o LEFT JOIN bikes b ON b.id = o.bike_id
         WHERE o.user_id = ? AND o.status = 'renting'
         ORDER BY o.id DESC LIMIT 1`,
        [ctx.user.id]
      );
      if (!rows.length) return { data: { hasActiveOrder: false }, summary: '███████████' };
      const o = rows[0];
      const minutes = Math.ceil((Date.now() - new Date(o.start_time).getTime()) / 60000);
      return {
        data: {
          hasActiveOrder: true,
          orderNo: o.order_no,
          bikeNo: o.bike_no,
          bikeName: o.bike_name,
          bikeType: o.bike_type,
          startTime: o.start_time,
          minutesUsed: minutes,
          hourlyRate: Number(o.hourly_rate),
          currentFare: calcFare(o.hourly_rate, minutes / 60).total
        },
        summary: `████${o.bike_no}███ ${minutes} ■■`
      };
    }
  },

  get_my_orders: {
    roles: ['student', 'admin', 'staff'],
    description: '■■"■"████████████████████████"██████""█████████"██████',
    parameters: {
```

```
      type: 'object',
      properties: { limit: { type: 'integer', description: '████████ 5███ 20' } }
    },
    async run(args, ctx) {
      const limit = clampLimit(args.limit);
      const [rows] = await pool.execute(
        `SELECT o.order_no, o.start_time, o.end_time, o.duration_hours, o.total_amount, o.status,
                b.bike_no
         FROM rental_orders o LEFT JOIN bikes b ON b.id = o.bike_id
         WHERE o.user_id = ?
         ORDER BY o.id DESC LIMIT ${limit}`,
        [ctx.user.id]
      );
      return {
        data: { count: rows.length, orders: rows },
        summary: `██ ${rows.length} ██████`
      };
    }
  },

  list_stations: {
    roles: ['student', 'admin', 'staff'],
    description: '████████████████████████"██████""██████"██████',
    parameters: { type: 'object', properties: {} },
    async run() {
      const [rows] = await pool.execute(
        `SELECT s.id, s.name_zh, s.name_en, s.address_zh, s.address_en, s.capacity,
                SUM(CASE WHEN b.status = 'available' THEN 1 ELSE 0 END) AS available_count,
                COUNT(b.id) AS bike_count
         FROM stations s LEFT JOIN bikes b ON b.station_id = s.id
         GROUP BY s.id, s.name_zh, s.name_en, s.address_zh, s.address_en, s.capacity
         ORDER BY s.id`
      );
      const totalAvailable = rows.reduce((a, r) => a + Number(r.available_count || 0), 0);
      return {
        data: { stationCount: rows.length, totalAvailable, stations: rows },
        summary: `${rows.length} ██████ ${totalAvailable} ████`
      };
    }
  },

  get_bike_info: {
    roles: ['student', 'admin', 'staff'],
    description: '████████████████████████████████████████"█████""█████████"██████',
    parameters: {
      type: 'object',
      properties: { bike_no: { type: 'string', description: '████████ BIKE-1001' } },
      required: ['bike_no']
    },
    async run(args) {
      const bikeNo = String(args.bike_no || '').trim();
      if (!bikeNo) return { ok: false, error: '██ bike_no ██' };
      const [rows] = await pool.execute(
        `SELECT b.bike_no, b.name, b.type, b.status, b.hourly_rate,
                s.name_zh AS station_name_zh, s.name_en AS station_name_en
         FROM bikes b LEFT JOIN stations s ON s.id = b.station_id
         WHERE b.bike_no = ? LIMIT 1`,
        [bikeNo]
      );
      if (!rows.length) return { ok: false, error: `████████ ${bikeNo} ████` };
      return { data: rows[0], summary: `${bikeNo}■${rows[0].status}` };
    }
  },

  estimate_rental_cost: {
    roles: ['student', 'admin', 'staff'],
    description: '████████████"■ 3 ██████""███████"████████████████ 1 ████ 1 ██████████',
    parameters: {
      type: 'object',
      properties: {
        hours: { type: 'number', description: '████████' },
        bike_no: { type: 'string', description: '████████████████████' },
        bike_type: { type: 'string', description: '███standard / city / sport' }
      },
      required: ['hours']
    },
    async run(args) {
      const hours = Number(args.hours);
      if (!Number.isFinite(hours) || hours <= 0) return { ok: false, error: 'hours ██████' };

      let rate = null;
      let basis = '';
      if (args.bike_no) {
        const [rows] = await pool.execute('SELECT hourly_rate FROM bikes WHERE bike_no = ? LIMIT 1', [String(args.bike_no).trim()]);
        if (rows.length) { rate = Number(rows[0].hourly_rate); basis = `██ ${args.bike_no}`; }
      }
      if (rate == null && args.bike_type) {
        const [rows] = await pool.execute('SELECT AVG(hourly_rate) AS r FROM bikes WHERE type = ?', [String(args.bike_type).trim()]);
        if (rows.length && rows[0].r != null) { rate = Number(rows[0].r); basis = `██ ${args.bike_type} ██████`; }
      }
      if (rate == null) {
```

```
      const [rows] = await pool.execute('SELECT MIN(hourly_rate) AS min_r, MAX(hourly_rate) AS max_r, AVG(hourly_rate) AS avg_r FROM bikes
      const r = rows[0] || {};
      if (r.min_r == null) return { ok: false, error: '[illegible]' };
      // [illegible]
      const lo = calcFare(r.min_r, hours).total;
      const hi = calcFare(r.max_r, hours).total;
      return {
        data: { hours, minTotal: lo, maxTotal: hi, note: '[illegible]' },
        summary: `${hours} [illegible] ${lo}~${hi} [illegible]`
      };
    }

    const { hours: billed, total } = calcFare(rate, hours);
    return {
      data: { hours: Number(hours), billedHours: billed, hourlyRate: rate, total, basis, rule: '[illegible]' },
      summary: `[illegible] ${rate} [illegible]/[illegible]${hours} [illegible] ${total} [illegible]`
    };
  }
},

list_announcements: {
  roles: ['student', 'admin', 'staff'],
  description: '[illegible]',
  parameters: {
    type: 'object',
    properties: { limit: { type: 'integer', description: '[illegible] 5[illegible] 20' } }
  },
  async run(args, ctx) {
    const limit = clampLimit(args.limit);
    const isAdmin = ctx.user?.role === 'admin';
    // [illegible] published —— [illegible]
    const where = isAdmin ? '' : "WHERE status = 'published'";
    const [rows] = await pool.execute(
      `SELECT title_zh, title_en, content_zh, content_en, status, created_at
       FROM announcements ${where} ORDER BY id DESC LIMIT ${limit}`
    );
    return {
      data: { count: rows.length, announcements: rows },
      summary: `[illegible] ${rows.length} [illegible]`
    };
  }
},

// [illegible] [illegible] + [illegible] [illegible]
admin_bike_status_summary: {
  roles: ['admin', 'staff'],
  description: '[illegible]/[illegible]/[illegible]"[illegible]""[illegible]"[illegible]',
  parameters: { type: 'object', properties: {} },
  async run() {
    const [byStatus] = await pool.execute('SELECT status, COUNT(*) AS n FROM bikes GROUP BY status');
    const [byType] = await pool.execute('SELECT type, COUNT(*) AS n FROM bikes GROUP BY type');
    const total = byStatus.reduce((a, r) => a + Number(r.n), 0);
    return {
      data: { total, byStatus, byType },
      summary: `[illegible] ${total} [illegible]`
    };
  }
},

// [illegible] [illegible] [illegible]
admin_revenue_stats: {
  roles: ['admin'],
  description: '[illegible]"[illegible]""[illegible]"[illegible]',
  parameters: { type: 'object', properties: {} },
  async run() {
    const [totals] = await pool.execute(
      `SELECT COUNT(*) AS orders, COALESCE(SUM(total_amount), 0) AS revenue,
              COALESCE(AVG(total_amount), 0) AS avg_amount
       FROM rental_orders WHERE status = 'completed'`
    );
    const [daily] = await pool.execute(
      `SELECT DATE(end_time) AS day, COUNT(*) AS orders, COALESCE(SUM(total_amount), 0) AS revenue
       FROM rental_orders
       WHERE status = 'completed' AND end_time >= now() - INTERVAL '7 days'
       GROUP BY DATE(end_time) ORDER BY day DESC`
    );
    const [active] = await pool.execute(
      `SELECT COUNT(*) AS n FROM rental_orders WHERE status = 'renting'`
    );
    return {
      data: {
        completedOrders: Number(totals[0].orders),
        totalRevenue: Number(totals[0].revenue),
        avgOrderAmount: Number(Number(totals[0].avg_amount).toFixed(2)),
        rentingNow: Number(active[0].n),
        last7Days: daily
      },
      summary: `[illegible] ${Number(totals[0].revenue).toFixed(2)} [illegible]`
    };
  }
},
```

```
  admin_user_orders: {
    roles: ['admin'],
    description: '■■■■■■■■■■■■■■■■■■■■■■"■■■■■■■■■"■■■■■■■■■■■■■■■■■■■ ID ■■',
    parameters: {
      type: 'object',
      properties: { username: { type: 'string', description: '■■■■■■■■' } },
      required: ['username']
    },
    async run(args) {
      const username = String(args.username || '').trim();
      if (!username) return { ok: false, error: '■■ username ■■' };
      // ■■ ■■■■■■■■ username ■ name■■■■ SELECT *■■■■ password ■■■■■■■■
      const [users] = await pool.execute(
        'SELECT id, username, name, role, status FROM users WHERE username = ? LIMIT 1',
        [username]
      );
      if (!users.length) return { ok: false, error: `■■■■■■■ ${username}` };
      const u = users[0];
      const [orders] = await pool.execute(
        `SELECT o.order_no, o.start_time, o.end_time, o.duration_hours, o.total_amount, o.status, b.bike_no
         FROM rental_orders o LEFT JOIN bikes b ON b.id = o.bike_id
         WHERE o.user_id = ? ORDER BY o.id DESC LIMIT 20`,
        [u.id]
      );
      const completed = orders.filter((o) => o.status === 'completed');
      return {
        data: {
          user: { username: u.username, name: u.name, role: u.role, status: u.status },
          orderCount: orders.length,
          completedCount: completed.length,
          totalSpent: Number(completed.reduce((a, o) => a + Number(o.total_amount || 0), 0).toFixed(2)),
          recentOrders: orders.slice(0, 5)
        },
        summary: `${username} ■ ${orders.length} ■■■`
      };
    }
  }
};

/** ■■■■■■■■■■■■■■■■ run ■■■■■■■■■■■■■ */
export function toolNamesForRole(role) {
  return toolsForRole(role).map((t) => t.name);
}

/**
 * ■■■■■■■■**■■■■■■■■■■■**■■■■■■■■■ 2 ■■■
 *
 * @returns {{ok: boolean, data?: any, error?: string, summary?: string}}
 */
export async function executeTool(name, args, ctx) {
  const def = TOOLS[name];
  if (!def) {
    console.warn('[ai][security] ■■■■■■■■■■■%s■user=%s role=%s■', name, ctx?.user?.username, ctx?.user?.role);
    return { ok: false, error: `■■■■ ${name} ■■■` };
  }

  // ■■■■■■■■ 2 ■■■■■■■■■■■■■■■■
  const role = ctx?.user?.role;
  if (!def.roles.includes(role)) {
    console.warn('[ai][security] ■■■■■■■■tool=%s user=%s role=%s', name, ctx?.user?.username, role);
    return { ok: false, error: '■■■■■■■■■■■■' };
  }

  try {
    const result = await def.run(args || {}, ctx);
    if (result && result.ok === false) return result;
    return { ok: true, ...result };
  } catch (e) {
    console.error('[ai] ■■■■■■■ tool=%s■%s', name, e.message);
    // ■■ SQL ■■■■■■■■■■■■■■■■■■■■
    return { ok: false, error: '■■■■■■■■■■■■■■■■■■■' };
  }
}
```

源码文件：frontend/src/App.vue

SHA-256：ae02acc453af7a354fbb7d2517e25632e52e6faf420a2caddcaac13f0724aa45

```
<template>
  <div class="app-shell">
    <header class="topbar">
      <router-link class="brand" to="/">{{ t('app') }}</router-link>
      <nav class="toolbar">
        <router-link to="/">{{ t('home') }}</router-link>
        <router-link v-if="auth.user" to="/bikes">{{ t('bikes') }}</router-link>
        <router-link v-if="auth.user" to="/orders/current">{{ t('currentOrder') }}</router-link>
        <router-link v-if="auth.user" to="/orders/history">{{ t('history') }}</router-link>
        <router-link to="/announcements">{{ t('announcements') }}</router-link>
```

```
        <router-link v-if="auth.user" to="/profile">{{ t('profile') }}</router-link>
        <router-link v-if="auth.user?.role === 'admin'" to="/admin">{{ t('dashboard') }}</router-link>
        <!-- staff ■■■ /admin■■■■ admin-only ■■■■■■■■■■■■■■■■■■■ -->
        <router-link v-if="auth.user?.role === 'staff'" to="/admin/maintenance">{{ t('maintenance') }}</router-link>
        <router-link v-if="!auth.user" to="/login">{{ t('login') }}</router-link>
        <button v-if="auth.user" @click="logout">{{ t('logout') }}</button>
        <select :value="state.lang" @change="setLang($event.target.value)">
          <option value="zh-CN">■■</option>
          <option value="en-US">English</option>
          <option value="ms-MY">Bahasa Melayu</option>
          <option value="vi-VN">Ti■ng Vi■t</option>
          <option value="lo-LA">■■■■■■■■</option>
        </select>
      </nav>
    </header>
    <main class="content">
      <router-view />
    </main>

    <!-- ■■■■■■■■■ + ■■■■■■ shell ■■■■■■■■■■■
         ■■■■■■■■■■■■■■■■■■■
```

```
router.beforeEach((to) => {
  if (to.meta.auth && !authStore.isAuthed) return '/login';
  if (to.meta.admin && authStore.user?.role !== 'admin') return '/';
  // roles ■■ admin ■■■■■■■■■■■■■■ staff
  if (to.meta.roles && !to.meta.roles.includes(authStore.user?.role)) return '/';
  return true;
});

export default router;
```

源码文件：frontend/src/stores/auth.js

SHA-256：934d47dbd2553075dd2a17975324bf4a15c848e6eb90b578048c13692caf8882

```
import { reactive } from 'vue';
import { request } from '../api/client.js';

export const authStore = reactive({
  user: JSON.parse(localStorage.getItem('user') || 'null'),
  get isAuthed() {
    return Boolean(localStorage.getItem('token'));
  },
  async login(form) {
    const data = await request('/auth/login', { method: 'POST', body: form });
    localStorage.setItem('token', data.token);
    localStorage.setItem('user', JSON.stringify(data.user));
    this.user = data.user;
  },
  async loginWithOidc(accessToken) {
    const data = await request('/auth/oidc', { method: 'POST', body: { accessToken } });
    localStorage.setItem('token', data.token);
    localStorage.setItem('user', JSON.stringify(data.user));
    this.user = data.user;
    return data.user;
  },
  async linkOidc(accessToken) {
    return request('/auth/oidc/link', { method: 'POST', body: { accessToken } });
  },
  async oidcLinked() {
    return request('/auth/oidc/status');
  },
  logout() {
    localStorage.removeItem('token');
    localStorage.removeItem('user');
    this.user = null;
  }
});
```

源码文件：frontend/src/stores/ai.js

SHA-256：454765c730ce853b18facc71e0ae09474dadaaef6bc3d27e40b8907f4055f550

```
/**
 * ■■■■■■■■■■■■■■ stores/auth.js ■■■■■reactive ■■ + localStorage■■
 * ■■■ Pinia ── ■■■■■■■■■■ store■■■■■■■■■■■■■
 *
 * ■■■■■■■■■
 *   1. ■■■■■■■■■■■■■■■■■■■■■■
 *   2. ■■■■■■**■■■■■■**■■■■■■■■■■■"■■■■■"■■■■■■
 *      ■■ localStorage ■■■■■■■■■■■■■■■■■■
 *   3. ■■■■■■■■■■■ → ■■■■■■■■■■■■■■■ → ■■■
 */
import { reactive } from 'vue';
import { request, streamRequest } from '../api/client.js';

/** ■■■■■■■■■■■■■■■■■■■
```

```
  catalogError: '',
  defaultModel: '',
  /** ■■■■■■■■ id■■■ `ollama:qwen3:14b` */
  model: localStorage.getItem(LS_MODEL) || '',
  /** ■■■■■■■"■■■■■■■■■■■■"■■■■■■■■■ */
  capabilities: [],

  // ■■ ■■ ■■
  messages: [],
  /** '' | 'connecting' | 'waiting' | 'streaming' */
  status: '',
  /** ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
  waited: 0,
  error: '',

  voiceEnabled: localStorage.getItem(LS_VOICE) === '1',

  /** ■■■■■■■ AbortController */
  _abort: null,
  _timer: null,

  get streaming() {
    return this.status === 'connecting' || this.status === 'waiting' || this.status === 'streaming';
  },

  /** ■■■■■■■■■■■■■■■■■■■■■■ */
  get flatModels() {
    return this.groups.flatMap((g) => (g.models || []).map((m) => ({ ...m, groupLabel: g.label })));
  },

  async loadCatalog({ force = false } = {}) {
    if (this.catalogLoaded && !force) return;
    try {
      const lang = localStorage.getItem('lang') || 'zh-CN';
      const data = await request(`/ai/models?lang=${encodeURIComponent(lang)}`);
      this.groups = data.groups || [];
      this.defaultModel = data.defaultModel || '';
      this.capabilities = data.capabilities || [];
      this.catalogError = '';

      // ■■■■■■■■■■■■■■■■■■■/■■■■■■■■■■
      const ids = this.flatModels.map((m) => m.id);
      if (!ids.includes(this.model)) this.setModel(ids.includes(this.defaultModel) ? this.defaultModel : ids[0] || '');

      this.catalogLoaded = true;
    } catch (e) {
      this.catalogError = e.message;
    }
  },

  setModel(id) {
    this.model = id || '';
    try { localStorage.setItem(LS_MODEL, this.model); } catch { /* ■■■■ */ }
  },

  setVoice(on) {
    this.voiceEnabled = Boolean(on);
    try { localStorage.setItem(LS_VOICE, this.voiceEnabled ? '1' : '0'); } catch { /* ■■■■ */ }
  },

  openPanel() {
    this.open = true;
    this.everOpened = true;
    this.loadCatalog();
  },

  closePanel() {
    this.open = false;
    // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    // ■■■■■■■■"■■"■■■■■■■■■
  },

  reset() {
    this.stop();
    this.messages = [];
    this.error = '';
    this.status = '';
  },

  /** ■■■■■■■■■■■■■■■■■■ abort ■■■ */
  stop() {
    if (this._abort) {
      try { this._abort.abort(); } catch { /* ■■■ */ }
      this._abort = null;
    }
    this._stopTimer();
    if (this.status) this.status = '';
  },

  _stopTimer() {
    if (this._timer) {
```

```
      clearInterval(this._timer);
      this._timer = null;
    }
  },

  _startTimer() {
    this._stopTimer();
    this.waited = 0;
    this._timer = setInterval(() => { this.waited += 1; }, 1000);
  },

  /**
   * ■■■■■■■■■■■
   * ■■■■■■ backend/src/ai/agent.js ── ■■■■■ type■■■■■■■■■■■■■■■
   */
  async send(text) {
    const content = String(text || '').trim();
    if (!content || this.streaming) return;
    if (!this.model) {
      this.error = 'aiPickModel';
      return;
    }

    this.error = '';
    this.messages.push({ role: 'user', content });

    // ■■■■■■■■■■■■■■■■■■■■■■■■■■"■■/■■"■■■■■
    const reply = reactive({ role: 'assistant', content: '', tools: [], error: '', stopped: false });
    this.messages.push(reply);

    const controller = new AbortController();
    this._abort = controller;
    this.status = 'connecting';
    this._startTimer();

    // ■■ user/assistant ■■■■■■■■■■■■■■■■■■■■■■
    const history = this.messages
      .filter((m) => !m.error && (m.role === 'user' || m.role === 'assistant'))
      .slice(0, -1)   // ■■■■■■■■■■■■■■■
      .slice(-MAX_HISTORY)
      .map((m) => ({ role: m.role, content: m.content }));

    try {
      for await (const ev of streamRequest('/ai/chat', {
        body: { model: this.model, messages: history, lang: localStorage.getItem('lang') || 'zh-CN' },
        signal: controller.signal
      })) {
        // ■■■■■■■■■■■■■■■■"■■■■"■
        if (ev.type === 'text' && !reply.content) {
          this.status = 'streaming';
          this._stopTimer();
        }
        if (ev.type === 'ping') {
          // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
          if (this.status === 'connecting') this.status = 'waiting';
          continue;
        }
        this._apply(ev, reply);
      }
    } catch (e) {
      // ■■■■■■■■■AbortError ■■■■■■■■■■■■■■
      if (e.name === 'AbortError' || controller.signal.aborted) {
        reply.stopped = true;
      } else {
        reply.error = e.message || 'aiFailed';
      }
    } finally {
      this._abort = null;
      this._stopTimer();
      this.status = '';
    }
  },

  /** ■■■■ canonical ■■■■■■■■■■■■■■ */
  _apply(ev, reply) {
    switch (ev.type) {
      case 'text':
        reply.content += ev.text;
        break;
      case 'tool_call':
        // ■■■■■■■■■■■■■ i18n■■ ChatWidget ■■ tool_* ■■
        reply.tools.push({ name: ev.name, done: false, ok: true });
        break;
      case 'tool_result': {
        const t = reply.tools.find((x) => x.name === ev.name && !x.done);
        if (t) { t.done = true; t.ok = ev.ok !== false; }
        break;
      }
      case 'reset':
        // ■■"■■■■■"■■■■■■■■■■■■■■■■■■■■■■
        reply.content = '';
```

```
        break;
      case 'error':
        reply.error = ev.message || 'aiFailed';
        break;
      case 'done':
        // ■■■■■■■■■■■■■■■■■■■■■■■■■■
        if (!reply.content && !reply.tools.length) reply.error = reply.error || 'aiEmpty';
        break;
      default:
        break;
    }
  }
});
```

源码文件：frontend/src/components/ChatWidget.vue

SHA-256：d5c9cf8f6ae625b37a931adcd163893f652f5c122e98c982d607d0c0ec86879a

```
<template>
  <!-- ■■■■■■■■■■■■■■■■ /ai/chat ■■■■■■■■■■■■■■■■■■ 401■ -->
  <button
    v-if="auth.user"
    class="chat-ball"
    type="button"
    :aria-label="t('aiTitle')"
    @click="ai.open ? ai.closePanel() : ai.openPanel()"
  >
    <span class="chat-ball-icon">{{ ai.open ? '×' : '■' }}</span>
  </button>

  <Teleport to="body">
    <section v-if="ai.open && auth.user" class="chat-panel" role="dialog" aria-modal="false">
      <header class="chat-head">
        <strong>{{ t('aiTitle') }}</strong>

        <select
          class="chat-model"
          :value="ai.model"
          :disabled="ai.streaming"
          :title="t('aiModel')"
          @change="ai.setModel($event.target.value)"
        >
          <option v-if="!ai.flatModels.length" value="">{{ t('aiNoModel') }}</option>
          <optgroup v-for="g in ai.groups" :key="g.id" :label="g.label">
            <option v-for="m in g.models" :key="m.id" :value="m.id">{{ label(m) }}</option>
          </optgroup>
        </select>

        <!-- ■■■■■■■■■■■■■■■■■■ HTTP ■■■■■■■■title ■■■■■■■
             ■■■■■■■■■■■■ -->
        <button
          class="chat-icon-btn"
          type="button"
          :class="{ active: dialogOn, disabled: !voiceReady }"
          :disabled="!voiceReady"
          :title="voiceReady ? t('aiDialog') : t(voiceBlockReason)"
          @click="toggleDialog"
        >{{ dialogOn ? '■' : '■' }}</button>

        <button
          class="chat-icon-btn"
          type="button"
          :class="{ active: ai.voiceEnabled }"
          :title="t('aiVoice')"
          @click="toggleVoice"
        >{{ ai.voiceEnabled ? '■' : '■' }}</button>

        <button class="chat-icon-btn" type="button" :title="t('aiClear')" @click="ai.reset()">■</button>
        <button class="chat-icon-btn" type="button" :title="t('close')" @click="ai.closePanel()">×</button>
      </header>

      <div ref="listEl" class="chat-body">
        <!-- ■■■■■■■■■■■■/■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ -->
        <p v-if="ai.catalogError" class="chat-note err">{{ t('aiFailed') }}</p>
        <p v-if="ai.error" class="chat-note err">{{ t(ai.error) }}</p>
        <p v-if="voiceError" class="chat-note err">{{ t(voiceError) }}</p>

        <!-- ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ -->
        <div v-if="dialogOn" class="chat-voice">
          <span class="chat-mic-dot" :class="voiceState"></span>
          <span>{{ t(voiceStateKey) }}</span>
        </div>

        <!-- ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ -->
        <div v-if="!ai.messages.length" class="chat-empty">
          <p class="muted">{{ t('aiIntro') }}</p>
          <ul class="chat-caps">
            <li v-for="c in ai.capabilities" :key="c">{{ c }}</li>
```

```
          </ul>
          <div class="chat-suggests">
            <button
              v-for="s in suggestions"
              :key="s"
              class="chat-suggest"
              type="button"
              @click="ai.send(s)"
            >{{ s }}</button>
          </div>
        </div>

        <div
          v-for="(m, i) in ai.messages"
          :key="i"
          class="chat-msg"
          :class="m.role"
        >
          <div class="chat-bubble">
            <!-- ■■■■■■■■■■ chip■■■■■■■"■■■■■"■■■"■■■■■" -->
            <div v-if="m.tools?.length" class="chat-tools">
              <span
                v-for="(tool, ti) in m.tools"
                :key="ti"
                class="chat-tool"
                :class="{ failed: tool.done && !tool.ok }"
              >
                {{ tool.done ? (tool.ok ? '✓' : '✕') : '■' }} {{ t('tool_' + tool.name) }}
              </span>
            </div>

            <p v-if="m.content" class="chat-text">{{ m.content }}</p>
            <p v-if="m.stopped" class="chat-note muted">{{ t('aiStopped') }}</p>
            <!-- m.error ■■■ i18n key■aiFailed…■■■■■■■■■■■■■
                 t() ■■■■■■■■■■ key■■■■■■■■■■■■■■■■■ -->
            <p v-if="m.error" class="chat-note err">{{ t(m.error) }}</p>
          </div>
        </div>

        <!-- ■■ token■■■■■■■■■■■■■■■"■■■■■"■■■ -->
        <div v-if="waiting" class="chat-waiting muted">
          <span class="chat-dots"><i></i><i></i><i></i></span>
          {{ ai.status === 'connecting' ? t('aiConnecting') : t('aiThinking') }}
          <template v-if="ai.waited >= 3">■{{ ai.waited }}s■</template>
        </div>
      </div>

      <footer class="chat-foot">
        <textarea
          ref="inputEl"
          v-model="draft"
          class="chat-input"
          rows="1"
          :placeholder="t('aiPlaceholder')"
          :disabled="ai.streaming"
          @keydown.enter.exact.prevent="submit"
        ></textarea>
        <button
          v-if="ai.streaming"
          class="btn chat-send stop"
          type="button"
          @click="ai.stop()"
        >{{ t('aiStop') }}</button>
        <button
          v-else
          class="btn chat-send"
          type="button"
          :disabled="!draft.trim()"
          @click="submit"
        >{{ t('aiSend') }}</button>
      </footer>
    </section>
  </Teleport>
</template>

<script setup>
import { ref, computed, watch, nextTick } from 'vue';
import { aiStore as ai } from '../stores/ai.js';
import { authStore as auth } from '../stores/auth.js';
import { t } from '../i18n/index.js';
import { speak, stopSpeaking, capabilities } from '../services/voice.js';
import { createDialogController, bindBridgeCallbacks } from '../services/dialog.js';

const draft = ref('');
const listEl = ref(null);
const inputEl = ref(null);

const waiting = computed(() => ai.status === 'connecting' || ai.status === 'waiting');

// ■■ ■■■■ ■■
const caps = capabilities();
```

```
const voiceReady = caps.stt;
const dialogOn = ref(false);
/** ■■■■■■■■■■■■■■■■■■■ */
const voiceState = ref('idle');
const voiceError = ref('');
/** ■■■■■■■■■■■■■■■■ SpeechRecognition■■■■■■■■ */
let dialog = null;

const voiceBlockReason = computed(() => (
  caps.reason === 'insecure-context' ? 'voiceInsecure' : 'voiceNoSR'
));

/** 'listening' → 'voiceListening'■■ dialog.js ■ stateKey ■■■■■■ */
const voiceStateKey = computed(
  () => 'voice' + voiceState.value.charAt(0).toUpperCase() + voiceState.value.slice(1)
);

function ensureDialog() {
  if (dialog) return dialog;
  dialog = createDialogController({
    onTranscript: (text) => {
      // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ chip ■■■■■
      ai.send(text);
      dialog.thinking();
    },
    onState: (s) => { voiceState.value = s; },
    onError: (code) => {
      voiceError.value = code === 'too-many-restarts' ? 'voiceTooManyRestarts' : 'voiceFailed';
      dialogOn.value = false;
    }
  });
  bindBridgeCallbacks(dialog);
  return dialog;
}

function toggleDialog() {
  if (dialogOn.value) {
    dialog?.stop();
    dialogOn.value = false;
    return;
  }
  voiceError.value = '';
  // ■■■■■■■■■■■■■■■■■■■■■■■■■■ —— ■■■■■■■■
  if (!ai.voiceEnabled) ai.setVoice(true);
  ensureDialog().start();
  dialogOn.value = true;
}

/** ■■■■■■■■■■■■qwen3:14b ■■■■ ollama:qwen3:14b */
function label(m) {
  return m.label || m.name || m.id;
}

const suggestions = computed(() => {
  const zh = (localStorage.getItem('lang') || 'zh-CN') === 'zh-CN';
  return zh
    ? ['■■■■■■■■■■', '■■■■■■■■■■', '■■■■■■■■■■■■']
    : ['How much does renting cost?', 'Which stations have bikes now?', 'What is my current order?'];
});

function submit() {
  const text = draft.value.trim();
  if (!text || ai.streaming) return;
  draft.value = '';
  ai.send(text);
}

function toggleVoice() {
  ai.setVoice(!ai.voiceEnabled);
  // ■■■■■■■■■■■■■■■■■■■■■■"■■■■■■■■"
  if (!ai.voiceEnabled) stopSpeaking();
}

/** ■■■■■■■■■■■■■ nextTick ■■■■■■ DOM ■■■■■■■■■ */
watch(
  () => ai.messages.map((m) => m.content.length).join(',') + '|' + ai.messages.length,
  async () => {
    await nextTick();
    if (listEl.value) listEl.value.scrollTop = listEl.value.scrollHeight;
  }
);

/**
 * ■■■■■■■■**■■■■■■■■■**■■■■■■■■■■■■■■status ■■ ''■■
 * ■■■■■■■——■■■■■■■■■■■■■■■■■■■■
 */
watch(
  () => ai.status,
  (now, before) => {
    if (now || !before) return;
    const last = ai.messages[ai.messages.length - 1];
```

```
    // ■■■■■■■■■■■■■■■■■■■■■■ listening ■■■■■■■■
    // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    if (last?.role === 'assistant' && (last.error || !last.content)) {
      if (dialogOn.value) {
        voiceError.value = 'voiceFailed';
        dialog?.fail();
        dialogOn.value = false;
      }
      return;
    }

    if (!ai.voiceEnabled) return;
    if (last?.role !== 'assistant') return;
    // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    // ■■■■■■■■■■■■■■■■■■■■■■■
    if (dialogOn.value) dialog?.finish(last.content || '');
    else if (last.content) speak(last.content);
  }
);

/**
 * ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
 * ■■■■■■■■■■■■■■■■ 20 ■■■■■■■■■■ ── ■■■■■■■■
 * ■■■■■■■■"■■"■"■■"■■■■■■■■
 */
watch(
  () => ai.messages[ai.messages.length - 1]?.content || '',
  (text) => {
    if (!dialogOn.value || !text) return;
    dialog?.feed(text);
  }
);

// ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
watch(
  () => ai.open,
  (open) => {
    if (!open && dialogOn.value) {
      dialog?.stop();
      dialogOn.value = false;
    }
  }
);

// ■■■■■ = ■■■■■■■■■■■■■■■■■■■■■■■■■■■■ thinking ■
watch(
  () => ai.messages.length,
  (len) => {
    if (!len && dialogOn.value) {
      dialog?.stop();
      dialogOn.value = false;
    }
  }
);

watch(
  () => ai.open,
  async (open) => {
    if (open) {
      await nextTick();
      inputEl.value?.focus();
    }
  }
);

// ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ services/voice.js ■
if (import.meta.env.DEV && !capabilities().tts) {
  console.info('[ai] ■■■■■■■■■■■■■■■■■■■■■■■■■■');
}
</script>
```

源码文件：frontend/src/views/Login.vue

SHA-256：79d91817604e5eeb4d7d4c4d4ca8cfca557862342e5315f2bcca53b84c6497fc

```
<template>
  <section>
    <h1>{{ t('login') }}</h1>
    <form class="form" @submit.prevent="submit">
      <label v-if="isDev" class="field dev-test-account">
        <span class="field-label">{{ t('devTestAccount') }}</span>
        <select v-model="selectedAccount" @change="pickAccount">
          <option value="">{{ t('pleaseSelect') }}</option>
          <optgroup :label="t('student')">
            <option v-for="account in studentAccounts" :key="account.username" :value="account.username">{{ account.username }}</option>
          </optgroup>
          <optgroup :label="t('admin')">
            <option v-for="account in adminAccounts" :key="account.username" :value="account.username">{{ account.username }}</option>
```

```
          </optgroup>
        </select>
      </label>
      <input v-model="form.username" :placeholder="t('username')" required />
      <input v-model="form.password" :placeholder="t('password')" type="password" required />
      <button class="btn">{{ t('login') }}</button>
      <button class="btn secondary oidc-login" type="button" @click="startOidcLogin">{{ t('unifiedLogin') }}</button>
      <div class="row">
        <router-link to="/register">{{ t('register') }}</router-link>
        <router-link to="/forgot-password">{{ t('forgotPassword') }}</router-link>
      </div>
      <p v-if="message" class="form-error">{{ message }}</p>
    </form>
  </section>
</template>
<script setup>
import { reactive, ref, onMounted } from 'vue';
import { useRouter } from 'vue-router';
import { authStore } from '../stores/auth.js';
import { t } from '../i18n/index.js';
import { beginOidcLogin } from '../auth/oidc.js';
const router = useRouter();
const message = ref('');
const form = reactive({ username: '', password: '' });
const selectedAccount = ref('');
// The local preview server is also used as the Android development target.
// Keep test credentials hidden on any non-local host, even if a preview build is used.
const isDev = import.meta.env.DEV
  || import.meta.env.VITE_SHOW_DEV_ACCOUNTS === '1'
  || ['localhost', '127.0.0.1', '::1'].includes(window.location.hostname);
const studentAccounts = Array.from({ length: 10 }, (_, index) => {
  const n = String(index + 1).padStart(2, '0');
  return { username: `student${n}`, password: `Student-${n}-Bike!2026` };
});
const adminAccounts = [
  { username: 'admin01', password: 'Admin-01-Bike!2026' },
  { username: 'admin02', password: 'Admin-02-Bike!2026' }
];
function pickAccount() {
  const account = [...studentAccounts, ...adminAccounts].find((item) => item.username === selectedAccount.value);
  if (account) Object.assign(form, { username: account.username, password: account.password });
}
onMounted(() => {
  message.value = localStorage.getItem('oidc-login-error') || '';
  localStorage.removeItem('oidc-login-error');
});

function startOidcLogin() {
  message.value = '';
  beginOidcLogin().catch((error) => { message.value = error.message; });
}

async function submit() {
  try {
    await authStore.login(form);
    router.push('/');
  } catch (e) {
    message.value = e.message;
  }
}
</script>
```

源码文件：frontend/src/views/Bikes.vue

SHA-256：ea2e8d2aac457f7e7bd08a8ac46c34176b249445fbd4e56151140601c692145a

```
<template>
  <section>
    <div class="row">
      <h1>{{ t('bikes') }}</h1>
      <input v-model="keyword" :placeholder="t('search')" @keyup.enter="search" />
      <select v-model="status" @change="search">
        <option value="">{{ t('all') }}</option>
        <option value="available">{{ t('available') }}</option>
        <option value="rented">{{ t('rented') }}</option>
        <option value="maintenance">{{ t('maintenanceStatus') }}</option>
      </select>
      <button class="btn secondary" @click="search">{{ t('search') }}</button>
    </div>
    <p v-if="error" class="form-error">{{ error }}</p>
    <p v-if="loading" class="muted">{{ t('loading') }}</p>
    <div class="grid">
      <article v-for="bike in bikes" :key="bike.id" class="card">
        <h3>{{ bike.name || bike.bike_no }}</h3>
        <p>{{ bike.bike_no }} · {{ bike.type }}</p>
        <p><span class="status" :class="bike.status">{{ t(bike.status) }}</span></p>
        <p class="muted">{{ bike.station_name_zh || bike.station_name_en }}</p>
        <p>■{{ bike.hourly_rate }}/h</p>
```

```
        <router-link class="btn" :to="`/bikes/${bike.id}`">{{ t('detail') }}</router-link>
      </article>
    </div>
    <p v-if="!loading && !bikes.length" class="muted">{{ t('noData') }}</p>
    <Pagination v-model:page="page" v-model:page-size="pageSize" :total="total" />
  </section>
</template>
<script setup>
import { onMounted, ref, watch } from 'vue';
import Pagination from '../components/Pagination.vue';
import { request } from '../api/client.js';
import { t } from '../i18n/index.js';
const bikes = ref([]);
const keyword = ref('');
const status = ref('');
const page = ref(1);
const pageSize = ref(10);
const total = ref(0);
const loading = ref(false);
const error = ref('');
let requestVersion = 0;

async function load() {
  const version = ++requestVersion;
  const params = new URLSearchParams();
  if (keyword.value) params.set('keyword', keyword.value);
  if (status.value) params.set('status', status.value);
  params.set('page', String(page.value));
  params.set('pageSize', String(pageSize.value));
  // ■■■■■■■■■■ { list, total, page, pageSize }■■■■■■■■
  loading.value = true;
  error.value = '';
  try {
    const data = await request(`/bikes?${params}`);
    if (version !== requestVersion) return;
    bikes.value = data?.list || [];
    total.value = Number(data?.total || 0);
    page.value = Number(data?.page || 1);
    pageSize.value = Number(data?.pageSize || pageSize.value);
  } catch (e) {
    if (version === requestVersion) error.value = e.message;
  } finally {
    if (version === requestVersion) loading.value = false;
  }
}

function search() {
  if (page.value === 1) load();
  else page.value = 1;
}

onMounted(load);
watch([page, pageSize], load);
</script>
```

源码文件：frontend/src/views/Orders.vue

SHA-256：0f108366307ac8705193d0e13f9d00ef1d2e5f6f832ac4445d2ad4e58d22bf96

```
<template>
  <section>
    <h1>{{ t('history') }}</h1>
    <p class="muted">{{ t('paymentMockNote') }}</p>
    <p v-if="error" class="form-error">{{ error }}</p>
    <p v-if="loading" class="muted">{{ t('loading') }}</p>
    <div class="card" style="overflow:auto">
      <table>
        <thead><tr><th>No.</th><th>Bike</th><th>Status</th><th>Hours</th><th>Total</th><th>{{ t('payment') }}</th></tr></thead>
        <tbody><tr v-for="o in orders" :key="o.id"><td>{{ o.order_no }}</td><td>{{ o.bike_id }}</td><td><span class="status" :class="o.statu
          <span v-if="paidOrders.has(Number(o.id))" class="status completed">{{ t('paymentSuccess') }}</span>
          <div v-else-if="o.status === 'completed'" class="row">
            <button class="btn" :disabled="paying === o.id" @click="pay(o, 'alipay')">{{ t('payAlipay') }}</button>
            <button class="btn secondary" :disabled="paying === o.id" @click="pay(o, 'wechat')">{{ t('payWechat') }}</button>
          </div>
        </td></tr></tbody>
        <tbody v-if="!loading && !orders.length"><tr><td colspan="5">{{ t('noData') }}</td></tr></tbody>
      </table>
    </div>
    <Pagination v-model:page="page" v-model:page-size="pageSize" :total="total" />
  </section>
</template>
<script setup>
import { onMounted, ref, watch } from 'vue';
import Pagination from '../components/Pagination.vue';
import { request } from '../api/client.js';
import { t } from '../i18n/index.js';
const orders = ref([]);
const page = ref(1);
```

```
const pageSize = ref(10);
const total = ref(0);
const loading = ref(false);
const error = ref('');
const paidOrders = ref(new Set());
const paying = ref(null);
let requestVersion = 0;

async function load() {
  const version = ++requestVersion;
  const params = new URLSearchParams({ page: String(page.value), pageSize: String(pageSize.value) });
  loading.value = true;
  error.value = '';
  try {
    const result = await request(`/orders/my?${params}`);
    if (version !== requestVersion) return;
    orders.value = result?.list || [];
    total.value = Number(result?.total || 0);
    page.value = Number(result?.page || 1);
    pageSize.value = Number(result?.pageSize || pageSize.value);
    const payments = await request('/payments/my');
    paidOrders.value = new Set((payments || []).map((payment) => Number(payment.order_id)));
  } catch (e) {
    if (version === requestVersion) error.value = e.message;
  } finally {
    if (version === requestVersion) loading.value = false;
  }
}

async function pay(order, provider) {
  paying.value = order.id;
  error.value = '';
  try {
    await request(`/payments/orders/${order.id}/pay`, { method: 'POST', body: { provider } });
    paidOrders.value = new Set([...paidOrders.value, Number(order.id)]);
  } catch (e) {
    error.value = e.message;
  } finally {
    paying.value = null;
  }
}

onMounted(load);
watch([page, pageSize], load);
</script>
```

源码文件：frontend/src/assets/style.css

SHA-256：cf6ef553631a90c129d14dd32e6037236cb9c310ea948d1bc329e5778e864abe

```
* { box-sizing: border-box; }
body { margin: 0; font-family: Arial, "Microsoft YaHei", sans-serif; background: #f6f7f9; color: #172033; }
a { color: inherit; text-decoration: none; }
button, input, select, textarea { font: inherit; }
.app-shell { max-width: 980px; min-height: 100vh; margin: 0 auto; background: #fff; }
.topbar { position: sticky; top: 0; z-index: 5; display: flex; gap: 10px; align-items: center; justify-content: space-between; padding: 12px
.brand { font-weight: 700; }
.toolbar { display: flex; gap: 8px; align-items: center; flex-wrap: wrap; }
.toolbar a, .toolbar button, .toolbar select { border: 0; border-radius: 6px; padding: 8px 10px; background: rgba(255,255,255,.12); color: #
.toolbar select option { color: #172033; }
.content { padding: 16px; }
.grid { display: grid; gap: 12px; grid-template-columns: repeat(auto-fit, minmax(220px, 1fr)); }
.card { border: 1px solid #e4e8ef; border-radius: 8px; padding: 14px; background: #fff; box-shadow: 0 1px 2px rgba(0,0,0,.04); }
.row { display: flex; gap: 10px; align-items: center; flex-wrap: wrap; }
.form { display: grid; gap: 12px; max-width: 480px; }
.form input, .form select, .form textarea { width: 100%; border: 1px solid #ccd4df; border-radius: 6px; padding: 10px; }
.btn { border: 0; border-radius: 6px; padding: 10px 12px; background: #0d6efd; color: #fff; cursor: pointer; }
.btn.secondary { background: #5c667a; }
.btn.danger { background: #c0392b; }
.muted { color: #687386; }
.status { display: inline-block; border-radius: 999px; padding: 3px 8px; font-size: 12px; background: #e9edf3; }
.status.available, .status.completed, .status.published { background: #dff5e8; color: #126b35; }
.status.rented, .status.renting, .status.processing { background: #fff3cd; color: #7b5800; }
.status.maintenance, .status.disabled, .status.hidden { background: #fde2e2; color: #9b1c1c; }
table { width: 100%; border-collapse: collapse; font-size: 14px; }
th, td { border-bottom: 1px solid #e7ebf0; padding: 10px; text-align: left; vertical-align: top; }
.btn:disabled { opacity: .55; cursor: not-allowed; }
.center { text-align: center; }
.form-error { margin: 0; color: #9b1c1c; font-size: 14px; }
.form-ok { margin: 0; color: #126b35; font-size: 14px; }
.model-picker { max-width: 480px; margin: 0 0 18px; }
.model-picker-head { display: flex; align-items: center; justify-content: space-between; gap: 12px; margin-bottom: 8px; }
.model-select-label select { max-width: 220px; border: 1px solid #ccd4df; border-radius: 5px; background: #fff; padding: 8px; }
.model-viewer { position: relative; height: 210px; overflow: hidden; border: 1px solid #dce4e7; border-radius: 6px; background: #f5f8f8; }
.model-viewer canvas { display: block; width: 100%; height: 100%; }
.model-loading { position: absolute; inset: 0; display: grid; place-items: center; color: #687386; }
.model-caption { margin: 6px 0 0; font-size: 13px; }
.sr-only { position: absolute; width: 1px; height: 1px; padding: 0; margin: -1px; overflow: hidden; clip: rect(0,0,0,0); white-space: nowrap
```

```
.oidc-login { width: 100%; }
.profile-oidc { margin-top: 16px; }

/* ■■ ■■■■■■■■■■ / ■■■■■■■■■■ */
.field { display: grid; gap: 4px; }
.field-label { font-size: 13px; color: #3b4557; }
.field .req { color: #c0392b; font-style: normal; }
.field-error { color: #9b1c1c; }
.form .field select { width: 100%; border: 1px solid #ccd4df; border-radius: 6px; padding: 10px; background: #fff; }

/* ■■ ■■■■ ■■ */
.filters { margin-bottom: 14px; }
.filters input, .filters select, .filters .filter-input {
  border: 1px solid #ccd4df; border-radius: 6px; padding: 8px 10px; background: #fff;
}
.filter-date { display: inline-flex; align-items: center; gap: 6px; font-size: 13px; color: #3b4557; }

/* ■■ ■■ ■■ */
.table-wrap { overflow-x: auto; }
.table-wrap td { max-width: 260px; word-break: break-word; }
.cell-actions { white-space: nowrap; }
.cell-actions .btn { padding: 6px 9px; font-size: 13px; margin-right: 4px; }

/* ■■ ■■ ■■ */
.pager { display: flex; gap: 12px; align-items: center; justify-content: space-between; flex-wrap: wrap; margin-top: 14px; font-size: 13px; }
.pager-pages { display: flex; gap: 4px; flex-wrap: wrap; }
.pager-btn { min-width: 32px; border: 1px solid #ccd4df; border-radius: 6px; padding: 6px 8px; background: #fff; color: #172033; cursor: poi
.pager-btn:disabled { opacity: .45; cursor: not-allowed; }
.pager-btn.active { background: #0d6efd; border-color: #0d6efd; color: #fff; }
.pager-size { display: inline-flex; align-items: center; gap: 6px; color: #687386; }
.pager-size select { border: 1px solid #ccd4df; border-radius: 6px; padding: 6px 8px; }

/* ■■ ■■ ■■ */
.modal-mask { position: fixed; inset: 0; z-index: 50; display: flex; align-items: center; justify-content: center; padding: 16px; background
.modal { width: 100%; max-height: 88vh; display: flex; flex-direction: column; background: #fff; border-radius: 10px; box-shadow: 0 12px 40p
.modal-head { display: flex; align-items: center; justify-content: space-between; gap: 12px; padding: 14px 16px; border-bottom: 1px solid #e
.modal-head h3 { margin: 0; font-size: 16px; }
.modal-x { border: 0; background: transparent; font-size: 22px; line-height: 1; color: #687386; cursor: pointer; }
.modal-body { padding: 16px; overflow-y: auto; }
.modal-foot { display: flex; gap: 8px; justify-content: flex-end; padding: 12px 16px; border-top: 1px solid #e7ebf0; }
/* ■■■■■■■■■■■■■■■■■■■■ */
.modal .form { max-width: none; }

@media (max-width: 680px) {
  .topbar { align-items: flex-start; flex-direction: column; }
  .dev-test-account { display: none; }
  .toolbar a, .toolbar button, .toolbar select { padding: 7px 8px; font-size: 13px; }
  th, td { padding: 8px 6px; font-size: 12px; }
  .pager { justify-content: center; }
  .content { padding: 12px; }
  .content h1 { margin-top: 4px; font-size: 24px; }
  .card { padding: 12px; }
  .card p { margin: 6px 0 0; font-size: 13px; }
}

/* ■■■■■■■■■■■■■■■■■■■■ ■■■■■■■■■ + ■■ ■■■■■■■■■■■■■■■■■■■■ */

.chat-ball {
  position: fixed; right: 22px; bottom: 22px; z-index: 60;
  width: 52px; height: 52px; border-radius: 50%; border: 0;
  background: #0d6efd; color: #fff; font-size: 22px; cursor: pointer;
  box-shadow: 0 6px 20px rgba(13,110,253,.42); transition: transform .15s;
}
.chat-ball:hover { transform: scale(1.07); }
.chat-ball-icon { line-height: 1; }

.chat-panel {
  position: fixed; right: 22px; bottom: 86px; z-index: 60;
  width: 380px; max-width: calc(100vw - 32px);
  height: 540px; max-height: calc(100vh - 130px);
  display: flex; flex-direction: column;
  background: #fff; border-radius: 12px; overflow: hidden;
  box-shadow: 0 16px 48px rgba(16,35,63,.28);
}
.chat-head {
  display: flex; align-items: center; gap: 6px;
  padding: 10px 12px; border-bottom: 1px solid #e7ebf0; background: #f7f9fc;
}
.chat-head strong { font-size: 14px; margin-right: auto; }
.chat-model {
  max-width: 132px; border: 1px solid #ccd4df; border-radius: 6px;
  padding: 4px 6px; font-size: 12px; background: #fff; color: #172033;
}
.chat-icon-btn {
  border: 0; background: transparent; cursor: pointer; font-size: 15px;
  line-height: 1; padding: 5px 6px; border-radius: 6px; color: #687386;
}
.chat-icon-btn:hover { background: #e9eef6; }
.chat-icon-btn.active { background: #dbe7ff; }
/* ■■■■■■■■■■■■■■■■■■ http ■■■■■■■■■■ */
.chat-icon-btn:disabled { opacity: .4; cursor: not-allowed; }
```

```
.chat-icon-btn:disabled:hover { background: transparent; }

.chat-body { flex: 1; overflow-y: auto; padding: 12px; background: #fff; }
.chat-empty { font-size: 13px; }
.chat-caps { margin: 8px 0 0; padding-left: 18px; color: #48546a; font-size: 12px; line-height: 1.9; }
.chat-suggests { display: flex; flex-wrap: wrap; gap: 6px; margin-top: 12px; }
.chat-suggest {
  border: 1px solid #ccd4df; background: #fff; color: #172033;
  border-radius: 14px; padding: 5px 10px; font-size: 12px; cursor: pointer;
}
.chat-suggest:hover { background: #f0f5ff; border-color: #0d6efd; color: #0d6efd; }

.chat-msg { display: flex; margin-bottom: 10px; }
.chat-msg.user { justify-content: flex-end; }
.chat-bubble {
  max-width: 86%; padding: 8px 11px; border-radius: 10px;
  font-size: 13px; line-height: 1.65; white-space: pre-wrap; word-break: break-word;
}
.chat-msg.user .chat-bubble { background: #0d6efd; color: #fff; border-bottom-right-radius: 3px; }
.chat-msg.assistant .chat-bubble { background: #f1f4f9; color: #172033; border-bottom-left-radius: 3px; }
.chat-text { margin: 0; }
.chat-note { margin: 6px 0 0; font-size: 12px; }
.chat-note.err { color: #c0392b; }

.chat-tools { display: flex; flex-wrap: wrap; gap: 5px; margin-bottom: 6px; }
.chat-tool {
  display: inline-flex; align-items: center; gap: 4px;
  background: #e4ecfb; color: #2b4d8f; border-radius: 10px;
  padding: 3px 8px; font-size: 11px;
}
.chat-tool.failed { background: #fdeaea; color: #b03a2e; }

.chat-waiting { display: flex; align-items: center; gap: 6px; font-size: 12px; padding: 2px 4px; }
.chat-dots { display: inline-flex; gap: 3px; }
.chat-dots i {
  width: 5px; height: 5px; border-radius: 50%; background: #9aa7bd;
  animation: chat-blink 1.2s infinite ease-in-out;
}
.chat-dots i:nth-child(2) { animation-delay: .2s; }
.chat-dots i:nth-child(3) { animation-delay: .4s; }
@keyframes chat-blink { 0%, 80%, 100% { opacity: .25; } 40% { opacity: 1; } }

/* ■■■■■■■■■■■■■■■■■■■■■■■■"■■"■■"■■" */
.chat-voice {
  display: flex; align-items: center; gap: 7px;
  font-size: 12px; color: #48546a; padding: 4px 2px;
}
.chat-mic-dot {
  width: 8px; height: 8px; border-radius: 50%; background: #9aa7bd; flex: none;
}
/* ■■=■■■■■■■=■■■■■■■■■■■■■■■"■■■■■■■■■" */
.chat-mic-dot.listening { background: #e74c3c; animation: chat-pulse 1.4s infinite ease-in-out; }
.chat-mic-dot.transcribing { background: #e6a23c; }
.chat-mic-dot.thinking { background: #0d6efd; }
.chat-mic-dot.speaking { background: #2ecc71; }
@keyframes chat-pulse { 0%, 100% { opacity: 1; transform: scale(1); } 50% { opacity: .35; transform: scale(.8); } }

.chat-foot { display: flex; gap: 8px; padding: 10px 12px; border-top: 1px solid #e7ebf0; background: #f7f9fc; }
.chat-input {
  flex: 1; resize: none; border: 1px solid #ccd4df; border-radius: 8px;
  padding: 8px 10px; font: inherit; font-size: 13px; max-height: 96px; background: #fff; color: #172033;
}
.chat-send { white-space: nowrap; }
.chat-send.stop { background: #c0392b; border-color: #c0392b; color: #fff; }

@media (max-width: 680px) {
  .chat-panel { right: 8px; left: 8px; bottom: 78px; width: auto; height: 70vh; }
  .chat-ball { right: 14px; bottom: 14px; }
}

/* ■■ ■■■■AI ■■■■ ■■ */
.ai-head { display: flex; align-items: center; justify-content: space-between; gap: 12px; }
.ai-head h1 { margin: 0; }

.gpu-row { margin-bottom: 10px; }
.gpu-line { display: flex; align-items: baseline; justify-content: space-between; gap: 10px; flex-wrap: wrap; }
.gpu-line .muted { font-size: 12px; }
.gpu-bar {
  height: 8px; border-radius: 999px; background: #e9edf3; overflow: hidden; margin-top: 6px;
}
.gpu-bar i { display: block; height: 100%; background: #0d6efd; transition: width .3s ease; }

.gpu-procs { width: 100%; border-collapse: collapse; margin-top: 10px; font-size: 12px; }
.gpu-procs th, .gpu-procs td {
  text-align: left; padding: 5px 8px; border-bottom: 1px solid #eef1f6;
}
.gpu-procs th { color: #687386; font-weight: 500; }
```

源码文件：android/app/src/main/java/com/xuelinhu/lanwebclient/MainActivity.java

SHA-256：15934e04ef320e34a046177debc5be67c1d1351dfe6d89dd134498f0a071a9e4

```
package com.xuelinhu.lanwebclient;

import android.Manifest;
import android.app.Activity;
import android.app.AlertDialog;
import android.content.pm.PackageManager;
import android.graphics.Color;
import android.net.Uri;
import android.os.Bundle;
import android.view.View;
import android.view.ViewGroup;
import android.view.WindowInsets;
import android.widget.Button;
import android.webkit.PermissionRequest;
import android.webkit.WebChromeClient;
import android.webkit.WebResourceRequest;
import android.webkit.WebSettings;
import android.webkit.WebView;
import android.webkit.WebViewClient;
import android.widget.EditText;
import android.widget.LinearLayout;
import android.widget.ProgressBar;
import android.widget.Toast;

public final class MainActivity extends Activity {
    private static final int AUDIO_PERMISSION_REQUEST = 41;
    private android.content.SharedPreferences preferences;
    private WebView webView;
    private ProgressBar progressBar;
    private PermissionRequest pendingPermissionRequest;
    private String serverUrl;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        getWindow().getDecorView().setSystemUiVisibility(View.SYSTEM_UI_FLAG_LIGHT_STATUS_BAR);
        preferences = getSharedPreferences("lan-web-client", MODE_PRIVATE);
        serverUrl = preferences.getString("serverUrl", BuildConfig.DEFAULT_URL);
        showWebView();
    }

    private void showWebView() {
        LinearLayout root = new LinearLayout(this);
        root.setOrientation(LinearLayout.VERTICAL);
        root.setBackgroundColor(Color.WHITE);
        root.setOnApplyWindowInsetsListener((view, insets) -> {
            int top;
            int bottom;
            if (android.os.Build.VERSION.SDK_INT >= android.os.Build.VERSION_CODES.R) {
                android.graphics.Insets safeInsets = insets.getInsets(
                        WindowInsets.Type.statusBars() | WindowInsets.Type.navigationBars());
                top = safeInsets.top;
                bottom = safeInsets.bottom;
            } else {
                top = insets.getSystemWindowInsetTop();
                bottom = insets.getSystemWindowInsetBottom();
            }
            view.setPadding(0, top, 0, bottom);
            return insets;
        });
        LinearLayout toolbar = new LinearLayout(this);
        toolbar.setOrientation(LinearLayout.HORIZONTAL);
        Button serverButton = new Button(this);
        serverButton.setText("■■■■");
        serverButton.setOnClickListener(view -> showServerDialog());
        Button refreshButton = new Button(this);
        refreshButton.setText("■■");
        refreshButton.setOnClickListener(view -> webView.reload());
        toolbar.addView(serverButton, new LinearLayout.LayoutParams(0, dp(48), 1));
        toolbar.addView(refreshButton, new LinearLayout.LayoutParams(0, dp(48), 1));
        root.addView(toolbar, new LinearLayout.LayoutParams(
                ViewGroup.LayoutParams.MATCH_PARENT, dp(48)));
        webView = new WebView(this);
        progressBar = new ProgressBar(this, null, android.R.attr.progressBarStyleHorizontal);
        progressBar.setMax(100);
        progressBar.setProgressTintList(android.content.res.ColorStateList.valueOf(Color.rgb(0, 121, 107)));
        root.addView(progressBar, new LinearLayout.LayoutParams(
                ViewGroup.LayoutParams.MATCH_PARENT, dp(3)));
        root.addView(webView, new LinearLayout.LayoutParams(
                ViewGroup.LayoutParams.MATCH_PARENT, 0, 1));
        setContentView(root);
        WebSettings settings = webView.getSettings();
```

```
        settings.setJavaScriptEnabled(true);
        settings.setDomStorageEnabled(true);
        settings.setMediaPlaybackRequiresUserGesture(false);
        settings.setAllowFileAccess(false);
        settings.setAllowContentAccess(false);
        settings.setMixedContentMode(WebSettings.MIXED_CONTENT_NEVER_ALLOW);
        webView.setWebViewClient(new WebViewClient() {
            @Override
            public boolean shouldOverrideUrlLoading(WebView view, WebResourceRequest request) {
                if (isSameOrigin(request.getUrl(), Uri.parse(serverUrl))) return false;
                Toast.makeText(MainActivity.this, "[redacted]", Toast.LENGTH_SHORT).show();
                return true;
            }

            @Override
            public android.webkit.WebResourceResponse shouldInterceptRequest(
                    WebView view, WebResourceRequest request) {
                Uri target = request.getUrl();
                if (("http".equals(target.getScheme()) || "https".equals(target.getScheme()))
                        && !isSameOrigin(target, Uri.parse(serverUrl))) {
                    return new android.webkit.WebResourceResponse(
                            "text/plain", "UTF-8", 403, "Blocked", null,
                            new java.io.ByteArrayInputStream(new byte[0]));
                }
                return null;
            }

            @Override
            public void onReceivedError(WebView view, WebResourceRequest request,
                                        android.webkit.WebResourceError error) {
                if (request.isForMainFrame()) {
                    Toast.makeText(MainActivity.this, "[redacted]", Toast.LENGTH_LONG).show();
                }
            }
        });
        webView.setWebChromeClient(new WebChromeClient() {
            @Override
            public void onProgressChanged(WebView view, int progress) {
                progressBar.setProgress(progress);
                progressBar.setVisibility(progress >= 100 ? ProgressBar.GONE : ProgressBar.VISIBLE);
            }

            @Override
            public void onPermissionRequest(PermissionRequest request) {
                if (!isSameOrigin(request.getOrigin(), Uri.parse(serverUrl))) {
                    request.deny();
                    return;
                }
                boolean requestsAudio = false;
                for (String resource : request.getResources()) {
                    requestsAudio |= PermissionRequest.RESOURCE_AUDIO_CAPTURE.equals(resource);
                    if (!PermissionRequest.RESOURCE_AUDIO_CAPTURE.equals(resource)) {
                        request.deny();
                        return;
                    }
                }
                if (requestsAudio && checkSelfPermission(Manifest.permission.RECORD_AUDIO)
                        != PackageManager.PERMISSION_GRANTED) {
                    pendingPermissionRequest = request;
                    requestPermissions(new String[]{Manifest.permission.RECORD_AUDIO}, AUDIO_PERMISSION_REQUEST);
                    return;
                }
                request.grant(request.getResources());
            }
        });
        webView.loadUrl(serverUrl);
    }

    @Override
    public void onRequestPermissionsResult(int requestCode, String[] permissions, int[] grantResults) {
        super.onRequestPermissionsResult(requestCode, permissions, grantResults);
        if (requestCode == AUDIO_PERMISSION_REQUEST && pendingPermissionRequest != null) {
            if (grantResults.length > 0 && grantResults[0] == PackageManager.PERMISSION_GRANTED) {
                pendingPermissionRequest.grant(new String[]{PermissionRequest.RESOURCE_AUDIO_CAPTURE});
            } else {
                pendingPermissionRequest.deny();
            }
            pendingPermissionRequest = null;
        }
    }

    private void showServerDialog() {
        EditText input = new EditText(this);
        input.setSingleLine(true);
        input.setHint("http://192.168.1.9:4026");
        input.setText(serverUrl);
        input.setSelectAllOnFocus(true);
        new AlertDialog.Builder(this)
                .setTitle("[redacted]")
                .setView(input)
                .setNegativeButton("[redacted]", null)
```

```
                .setPositiveButton("■■", (dialog, which) -> {
                    String candidate = input.getText().toString().trim().replaceAll("/+$", "");
                    if (!isAllowedServer(candidate)) {
                        Toast.makeText(this, "HTTP ■■■■■■■■■■■■■■■■ HTTPS", Toast.LENGTH_LONG).show();
                        return;
                    }
                    serverUrl = candidate;
                    preferences.edit().putString("serverUrl", serverUrl).apply();
                    webView.loadUrl(serverUrl);
                })
                .show();
    }

    private boolean isAllowedServer(String value) {
        Uri uri = Uri.parse(value);
        String scheme = uri.getScheme();
        String host = uri.getHost();
        if (host == null || uri.getUserInfo() != null
                || uri.getPort() != -1 && (uri.getPort() < 1 || uri.getPort() > 65535)) return false;
        if ("https".equalsIgnoreCase(scheme)) return true;
        if (!"http".equalsIgnoreCase(scheme)) return false;
        String[] octets = host.split("\\.");
        if ("localhost".equalsIgnoreCase(host)) return true;
        if (octets.length != 4) return false;
        try {
            int first = Integer.parseInt(octets[0]);
            int second = Integer.parseInt(octets[1]);
            int third = Integer.parseInt(octets[2]);
            int fourth = Integer.parseInt(octets[3]);
            if (first > 255 || second > 255 || third > 255 || fourth > 255) return false;
            return first == 10 || first == 127 || first == 192 && second == 168
                    || first == 172 && second >= 16 && second <= 31;
        } catch (NumberFormatException error) {
            return false;
        }
    }

    private boolean isSameOrigin(Uri target, Uri origin) {
        return target.getScheme() != null && target.getScheme().equals(origin.getScheme())
                && target.getHost() != null && origin.getHost() != null
                && target.getHost().equalsIgnoreCase(origin.getHost())
                && target.getPort() == origin.getPort();
    }

    private int dp(int value) {
        return Math.round(value * getResources().getDisplayMetrics().density);
    }

    @Override
    public void onBackPressed() {
        if (webView != null && webView.canGoBack()) webView.goBack();
        else super.onBackPressed();
    }

    @Override
    protected void onDestroy() {
        if (webView != null) webView.destroy();
        super.onDestroy();
    }
}
```

源码文件：backend/postgres/schema.sql

SHA-256：6b1ac542a66c334f019ee1f023f0d2cc3a002a4e0b452e01c49eef1328f2d05c

```
CREATE TABLE users (
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  username VARCHAR(50) NOT NULL,
  password VARCHAR(255) NOT NULL,
  name VARCHAR(100),
  email VARCHAR(120),
  phone VARCHAR(30),
  role TEXT NOT NULL DEFAULT 'student' CHECK (role IN ('student', 'admin', 'staff')),
  student_no VARCHAR(50),
  nationality VARCHAR(80),
  language VARCHAR(20) DEFAULT 'zh-CN',
  status TEXT NOT NULL DEFAULT 'active' CHECK (status IN ('active', 'disabled')),
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE stations (
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  name_zh VARCHAR(120) NOT NULL,
  name_en VARCHAR(120) NOT NULL,
  address_zh VARCHAR(255),
  address_en VARCHAR(255),
  latitude NUMERIC(10, 7),
```

```
  longitude NUMERIC(10, 7),
  capacity INTEGER NOT NULL DEFAULT 0 CHECK (capacity >= 0),
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE bikes (
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  bike_no VARCHAR(60) NOT NULL UNIQUE,
  name VARCHAR(120),
  type VARCHAR(60),
  status TEXT NOT NULL DEFAULT 'available' CHECK (status IN ('available', 'rented', 'maintenance', 'disabled')),
  station_id BIGINT REFERENCES stations(id) ON DELETE SET NULL,
  hourly_rate NUMERIC(10, 2) NOT NULL DEFAULT 2.00 CHECK (hourly_rate >= 0),
  image_url VARCHAR(500),
  description TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE rental_orders (
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  order_no VARCHAR(80) NOT NULL UNIQUE,
  user_id BIGINT NOT NULL REFERENCES users(id),
  bike_id BIGINT NOT NULL REFERENCES bikes(id),
  start_time TIMESTAMPTZ NOT NULL,
  end_time TIMESTAMPTZ,
  duration_hours INTEGER CHECK (duration_hours IS NULL OR duration_hours > 0),
  hourly_rate NUMERIC(10, 2) NOT NULL DEFAULT 2.00 CHECK (hourly_rate >= 0),
  total_amount NUMERIC(10, 2) DEFAULT 0 CHECK (total_amount IS NULL OR total_amount >= 0),
  status TEXT NOT NULL DEFAULT 'renting' CHECK (status IN ('renting', 'completed', 'cancelled')),
  start_station_id BIGINT,
  end_station_id BIGINT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE maintenance_records (
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  bike_id BIGINT NOT NULL REFERENCES bikes(id),
  staff_id BIGINT REFERENCES users(id),
  content TEXT NOT NULL,
  status TEXT NOT NULL DEFAULT 'processing' CHECK (status IN ('processing', 'finished')),
  start_time TIMESTAMPTZ,
  end_time TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE announcements (
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  title_zh VARCHAR(200) NOT NULL,
  title_en VARCHAR(200) NOT NULL,
  content_zh TEXT NOT NULL,
  content_en TEXT NOT NULL,
  status TEXT NOT NULL DEFAULT 'published' CHECK (status IN ('published', 'hidden')),
  created_by BIGINT REFERENCES users(id) ON DELETE SET NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_orders_user_status ON rental_orders (user_id, status);
CREATE INDEX idx_orders_bike ON rental_orders (bike_id);
CREATE INDEX idx_orders_start_station ON rental_orders (start_station_id);
CREATE INDEX idx_orders_end_station ON rental_orders (end_station_id);
CREATE INDEX idx_bikes_station ON bikes (station_id);
CREATE INDEX idx_maintenance_bike ON maintenance_records (bike_id);
CREATE INDEX idx_maintenance_staff ON maintenance_records (staff_id);
CREATE INDEX idx_announcements_created_by ON announcements (created_by);
CREATE UNIQUE INDEX uniq_users_username_casefold ON users (lower(username));

CREATE FUNCTION touch_updated_at() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
  NEW.updated_at = now();
  RETURN NEW;
END;
$$;

CREATE TRIGGER users_touch_updated_at BEFORE UPDATE ON users FOR EACH ROW EXECUTE FUNCTION touch_updated_at();
CREATE TRIGGER stations_touch_updated_at BEFORE UPDATE ON stations FOR EACH ROW EXECUTE FUNCTION touch_updated_at();
CREATE TRIGGER bikes_touch_updated_at BEFORE UPDATE ON bikes FOR EACH ROW EXECUTE FUNCTION touch_updated_at();
CREATE TRIGGER rental_orders_touch_updated_at BEFORE UPDATE ON rental_orders FOR EACH ROW EXECUTE FUNCTION touch_updated_at();
CREATE TRIGGER maintenance_records_touch_updated_at BEFORE UPDATE ON maintenance_records FOR EACH ROW EXECUTE FUNCTION touch_updated_at();
CREATE TRIGGER announcements_touch_updated_at BEFORE UPDATE ON announcements FOR EACH ROW EXECUTE FUNCTION touch_updated_at();
```

源码文件：backend/postgres/migrations/005_models_payments_i18n.js

SHA-256：59f4cd92f876087f1f70c834291502c5230f58e7350b0cd9704c847a5fef630c

```
export default {
  id: '005_models_payments_i18n',
  description: 'Published 3D bike models, locale constraints and mock payment ledger',
  async up(connection) {
    await connection.query(`
      CREATE TABLE bike_models (
        id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
        model_key VARCHAR(60) NOT NULL UNIQUE,
        name VARCHAR(120) NOT NULL,
        description TEXT NOT NULL DEFAULT '',
        preview_type VARCHAR(40) NOT NULL DEFAULT 'bike-city',
        color CHAR(7) NOT NULL DEFAULT '#128c75' CHECK (color ~ '^#[0-9A-Fa-f]{6}$'),
        status TEXT NOT NULL DEFAULT 'draft' CHECK (status IN ('published', 'draft')),
        sort_order INTEGER NOT NULL DEFAULT 0,
        created_by BIGINT REFERENCES users(id) ON DELETE SET NULL,
        created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
        updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
      );
      ALTER TABLE users ADD COLUMN preferred_model_id BIGINT REFERENCES bike_models(id) ON DELETE SET NULL;
      UPDATE users SET language = 'en-US' WHERE language NOT IN ('ms-MY', 'en-US', 'vi-VN', 'lo-LA');
      ALTER TABLE users ALTER COLUMN language SET DEFAULT 'en-US';
      ALTER TABLE users DROP CONSTRAINT IF EXISTS users_language_check;
      ALTER TABLE users ADD CONSTRAINT users_language_check CHECK (language IN ('ms-MY', 'en-US', 'vi-VN', 'lo-LA'));
      CREATE INDEX idx_bike_models_status_order ON bike_models (status, sort_order, id);
      CREATE TABLE payment_provider_configs (
        provider VARCHAR(20) PRIMARY KEY CHECK (provider IN ('alipay', 'wechat')),
        mode VARCHAR(10) NOT NULL DEFAULT 'mock' CHECK (mode = 'mock'),
        enabled BOOLEAN NOT NULL DEFAULT true,
        merchant_id_masked VARCHAR(80) NOT NULL DEFAULT '[redacted]',
        app_id_masked VARCHAR(80) NOT NULL DEFAULT '[redacted]',
        secret_env_name VARCHAR(80) NOT NULL,
        updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
      );
      CREATE TABLE payment_transactions (
        id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
        payment_no VARCHAR(80) NOT NULL UNIQUE,
        order_id BIGINT NOT NULL REFERENCES rental_orders(id) ON DELETE RESTRICT,
        user_id BIGINT NOT NULL REFERENCES users(id) ON DELETE RESTRICT,
        provider VARCHAR(20) NOT NULL REFERENCES payment_provider_configs(provider),
        mode VARCHAR(10) NOT NULL DEFAULT 'mock' CHECK (mode = 'mock'),
        amount NUMERIC(10, 2) NOT NULL CHECK (amount >= 0),
        currency CHAR(3) NOT NULL DEFAULT 'CNY',
        status TEXT NOT NULL DEFAULT 'succeeded' CHECK (status IN ('succeeded', 'failed')),
        provider_reference VARCHAR(100) NOT NULL,
        metadata JSONB NOT NULL DEFAULT '{}'::jsonb,
        created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
        UNIQUE (order_id)
      );
      CREATE INDEX idx_payment_transactions_user_time ON payment_transactions (user_id, created_at DESC);
      CREATE TRIGGER bike_models_touch_updated_at BEFORE UPDATE ON bike_models FOR EACH ROW EXECUTE FUNCTION touch_updated_at();
      INSERT INTO bike_models (model_key, name, description, preview_type, color, status, sort_order) VALUES
        ('city-commuter', 'City Commuter', 'Lightweight city bicycle with a clean commuter frame.', 'bike-city', '#128c75', 'published', 10),
        ('campus-classic', 'Campus Classic', 'Comfort-focused classic bicycle for campus routes.', 'bike-classic', '#e27a3f', 'published', 20),
        ('sport-hybrid', 'Sport Hybrid', 'Responsive hybrid bicycle with a vivid sport finish.', 'bike-sport', '#3478c8', 'published', 30);
      INSERT INTO payment_provider_configs (provider, mode, enabled, merchant_id_masked, app_id_masked, secret_env_name) VALUES
        ('alipay', 'mock', true, '[redacted]', '[redacted]', 'ALIPAY_PRIVATE_KEY'),
        ('wechat', 'mock', true, '[redacted]', '[redacted]', 'WECHAT_MCH_KEY');
    `);
  }
};
```